Python Lab Manual

```
def svd_jacobi(A):
  W, V = A.copy(), eye(n)
  for p < q:
    rotate(W[:, p], W[:, q])
  s = column_norms(W)
  return W / s, s, V.T
```

The SVD and the pseudo-inverse, implemented from scratch

Part 2 of a three-part course

1 **Textbook** — theory, worked hand calculations, exercises with full solutions.

2 **Python Lab Manual** (this volume) — every algorithm implemented from scratch in Python, with real output.

3 **Coding Assignments** — six graded assignments with an automatic `pytest` test suite.

Every listing in this manual is the actual source of the accompanying code, and every output box is what that code printed when this manual was built. Every figure was produced by the scripts shown.

Edition of October 2026

About This Manual

The textbook taught you to compute an SVD with pen and paper. This manual teaches you to compute one with a program — and to understand why the program has to be smarter than the pen-and-paper method. We build a small linear-algebra library, `linalg_scratch.py`, one function at a time: dot products and norms, Gaussian elimination, Gram–Schmidt and Householder QR, the power method and the Jacobi eigenvalue algorithm, two SVD algorithms, the pseudo-inverse, three least-squares solvers, regularisation, low-rank approximation and PCA.

The rules: what "from scratch" means

- Inside `linalg_scratch.py` we use NumPy only as a container for numbers: arrays, slicing, element-wise arithmetic, `sum`, `sqrt`, and — after we have written matrix multiplication ourselves in Part 1 — the `@` operator.
- **Nothing from `numpy.linalg` (or `scipy.linalg`) is used inside the library.** No `svd`, `eig`, `qr`, `solve`, `inv`, `pinv`, `lstsq`, not even `norm`.
- The demo scripts *do* call `numpy.linalg`, but only to check our answers. Being able to check your own code against a trusted implementation is a skill in itself.

Setting up

You need Python 3.10 or newer and three packages:

```
python -m pip install numpy matplotlib pytest
```

Unzip `svd_lab_code.zip`. You get this folder:

```
lab/
  linalg_scratch.py     the library we build in this manual
  plotstyle.py          shared Matplotlib settings for the figures
  run_demos.py          runs every demo, saving output to out/ and figures to figs/
  demos/                one script per experiment: d00_float.py ... d07_pca.py
```

Run any demo from inside the `lab` folder, for example

```
cd lab
python -m demos.d04_svd_examples
```

The `-m` form puts the `lab` folder on Python's import path, so the demos can import `linalg_scratch`. `python run_demos.py` runs all of them (about 20 seconds).

How to work through it

Each part follows the same pattern: a short recap of the mathematics (with a pointer to the textbook), the code, a demo that reproduces a worked example from the textbook, and an experiment that shows how the algorithm behaves in floating point. The parts match the six-week study plan:

Lab part	Textbook chapters	Assignment
0 Floating point	—	—
1 Vectors, matrices, elimination	1–2	A1
2 Orthogonalisation and QR	3	A2
3 Eigenvalues of symmetric matrices	4–5	A3
4 The SVD	6–7	A4
5 The pseudo-inverse	9	A5
6 Least squares and regularisation	8, 10	A5, A6
7 Applications: norms, low rank, PCA	7, 10	A6

Type the code in yourself. Reading code is not the same as writing it. A good routine is: read the section, close the manual, write the function from memory, run the demo, and only then compare with the listing. The Coding Assignments ask you to do exactly this, with automatic tests.

Contents

About This Manual i

0 Floating-Point Arithmetic 1

1 Vectors, Matrices and Elimination 3

1.1 Dot products and norms . . . 3

1.2 Matrix multiplication . . . 5

1.3 Gaussian elimination: rank and null space . . . 6

2 Orthogonalisation and QR 9

2.1 Two versions of Gram–Schmidt . . . 9

2.2 Householder QR . . . 11

2.3 How much orthogonality survives? . . . 12

2.4 Triangular systems, solving, projections . . . 13

3 Eigenvalues of Symmetric Matrices 16

3.1 Power iteration and deflation . . . 16

3.2 The Jacobi eigenvalue algorithm . . . 18

4 The Singular Value Decomposition 22

4.1 Method 1: the textbook recipe . . . 22

4.2 Method 2: one-sided Jacobi . . . 23

4.3 Checking against the worked examples . . . 24

4.4 Accuracy: why libraries don't form $A^{\mathsf{T}}A$ . . . 26

5 The Pseudo-Inverse 28

5.1 Implementation . . . 28

5.2 The tolerance matters . . . 30

6 Least Squares and Regularisation 32

6.1 Three solvers . . . 32

6.2 When the methods disagree . . . 33

6.3 Regularising an ill-posed problem . . . 35

7 Applications: Norms, Low Rank, PCA **38**

7.1 Norms, condition number, rank 38

7.2 Low-rank approximation of an image 39

7.3 Principal component analysis 42

Where Next: the Assignments **44**

A Complete Source of `linalg_scratch.py` **45**

B Helper Scripts **58**

CHAPTER 0

Floating-Point Arithmetic

Computers store real numbers in *binary floating point*: about 16 significant decimal digits and an exponent. Almost every surprising behaviour in numerical linear algebra comes from this one fact, so we start here.

demos/d00_float.py

```python
import numpy as np
from linalg_scratch import norm

print("0.1 + 0.2 == 0.3 ?", 0.1 + 0.2 == 0.3)
print("0.1 + 0.2       =", repr(0.1 + 0.2))
print("machine epsilon =", np.finfo(float).eps)
print("1 + 1e-16 == 1 ?", 1 + 1e-16 == 1)        # below eps/2: lost entirely
print("isclose(0.1+0.2, 0.3):", np.isclose(0.1 + 0.2, 0.3))

# catastrophic cancellation: subtracting nearly equal numbers
x = 1e-8
print("(1+x) - 1 =", (1 + x) - 1, " (exact answer 1e-8)")

# overflow in a naive length computation, and the scaled version
v = np.array([3e200, 4e200])
with np.errstate(over="ignore"):
    naive = np.sqrt(np.sum(v ** 2))
print("naive length :", naive)
print("scaled norm  :", norm(v))
```

OUTPUT

```
0.1 + 0.2 == 0.3 ? False
0.1 + 0.2       = 0.30000000000000004
machine epsilon = 2.220446049250313e-16
1 + 1e-16 == 1 ? True
isclose(0.1+0.2, 0.3): True
(1+x) - 1 = 9.99999993922529e-09  (exact answer 1e-8)
naive length : inf
scaled norm  : 4.9999999999999995e+200
```

Four lessons, which we will meet again and again:

1. **Most decimals are not exact.** 0.1 has no finite binary expansion, so `0.1 + 0.2` is not exactly `0.3`. *Never compare floats with* `==`; use a tolerance (`np.isclose`, `np.allclose`).
2. **Machine epsilon** $\varepsilon_{\text{mach}} = 2^{-52} \approx 2.2 \times 10^{-16}$ is the gap between 1 and the next larger float. Every arithmetic operation may commit a relative error of up to $\varepsilon_{\text{mach}}/2$. Adding 10^{-16} to 1 does nothing at all.
3. **Subtracting nearly equal numbers loses digits.** $(1 + 10^{-8}) - 1$ has only about 8 correct

digits: the leading digits cancel and what remains is mostly rounding error. This *catastrophic cancellation* is why the Läuchli example (textbook, Worked Example 10.5) breaks the $A^{\mathsf{T}}A$ approach.

4. **Overflow is avoidable.** $(3 \times 10^{200})^2$ overflows to `inf` even though the length 5×10^{200} is perfectly representable. Rescaling before squaring (Part 1) fixes this.

A rule of thumb used throughout: if a problem has condition number κ, a good ("backward stable") algorithm produces answers with relative error around $\kappa\,\varepsilon_{\text{mach}}$. You lose about $\log_{10}\kappa$ of your 16 digits, and no algorithm can do much better, because merely storing the data in floating point already perturbs it by $\varepsilon_{\text{mach}}$.

CHAPTER 1

Vectors, Matrices and Elimination

Textbook: Chapters 1–2. **Assignment:** A1.

1.1 Dot products and norms

The dot product is a loop that multiplies and adds. We write the loop explicitly once, so that nothing is hidden:

```
def dot(u, v):
    """Dot product u . v = sum_i u_i v_i, written as an explicit loop."""
    u = np.asarray(u, dtype=float).ravel()
    v = np.asarray(v, dtype=float).ravel()
    if u.shape != v.shape:
        raise ValueError(f"shape mismatch: {u.shape} vs {v.shape}")
    total = 0.0
    for ui, vi in zip(u, v):
        total += ui * vi
    return total
```

For the 2-norm we do one thing a naive implementation would not: divide by $s = \max_i |x_i|$ before squaring, then multiply back. Mathematically $s\sqrt{\sum (x_i/s)^2} = \sqrt{\sum x_i^2}$; numerically, the scaled entries are at most 1 in size and cannot overflow.

```
def norm(x, p=2):
    """Vector norm: p = 1, 2 or np.inf.

    The 2-norm is computed as  s * sqrt(sum((x/s)^2))  with s = max|x_i|.
    Mathematically this is just sqrt(sum x_i^2); numerically, the scaling
    stops x_i^2 from overflowing when |x_i| is huge (e.g. 1e200).
    """
    x = np.asarray(x, dtype=float).ravel()
    if x.size == 0:
        return 0.0
    if p == 1:
        return float(np.sum(np.abs(x)))
    if p == np.inf:
        return float(np.max(np.abs(x)))
    if p == 2:
        s = np.max(np.abs(x))
        if s == 0.0:
            return 0.0
        return float(s * np.sqrt(np.sum((x / s) ** 2)))
    raise ValueError("p must be 1, 2 or np.inf")
```

Normalising, angles and projection onto a line follow directly from the formulas of text-

book Chapter 1. Note the `np.clip` in `angle`: rounding can make the computed cosine 1.0000000000000002, and `arccos` of that is `nan`.

```
def normalize(x):
    """Unit vector x / ||x||.  Raises ValueError for the zero vector."""
    x = np.asarray(x, dtype=float)
    n = norm(x)
    if n == 0.0:
        raise ValueError("cannot normalise the zero vector")
    return x / n


def angle(u, v):
    """Angle between u and v in radians, in [0, pi]."""
    c = dot(u, v) / (norm(u) * norm(v))
    return float(np.arccos(np.clip(c, -1.0, 1.0)))   # clip guards rounding


def project_onto_vector(b, a):
    """Projection of b onto the line through a:  (a.b / a.a) a."""
    a = np.asarray(a, dtype=float)
    return (dot(a, b) / dot(a, a)) * a
```

demos/d01_vectors.py

```
import numpy as np
from linalg_scratch import dot, norm, normalize, angle, project_onto_vector

u, v = np.array([1, 2, 2]), np.array([2, 0, 1])          # Worked Example 1.1
print("u.v       =", dot(u, v))
print("|u|, |v|  =", norm(u), norm(v))
print("angle     = %.4f degrees" % np.degrees(angle(u, v)))
print("u/|u|     =", normalize(u))

x = np.array([3, -4, 0])                                  # Worked Example 1.2
print("norms 1,2,inf:", norm(x, 1), norm(x, 2), norm(x, np.inf))

a, b = np.array([1, 2, 2]), np.array([1, 1, 1])          # Worked Example 1.4
p = project_onto_vector(b, a)
e = b - p
print("p =", p, " (= 5/9 * a)")
print("e =", e, " e.a =", dot(e, a))

# compare with NumPy (allowed in demos, never inside linalg_scratch)
assert np.isclose(dot(u, v), u @ v)
assert np.isclose(norm(x), np.linalg.norm(x))
print("all checks against NumPy passed")
```

OUTPUT

```
u.v       = 4.0
|u|, |v|  = 3.0 2.23606797749979
angle     = 53.3957 degrees
u/|u|     = [0.33333333 0.66666667 0.66666667]
norms 1,2,inf: 7.0 5.0 4.0
```

```
p = [0.55555556 1.11111111 1.11111111]  (= 5/9 * a)
e = [ 0.44444444 -0.11111111 -0.11111111]  e.a = -2.220446049250313e-16
all checks against NumPy passed
```

Every number agrees with Worked Examples 1.1, 1.2 and 1.4 of the textbook. Look at `e.a`: it is -2.2×10^{-16}, not 0. The error vector is orthogonal to $\boldsymbol{a}$ *up to rounding*, which is the best a computer can do.

1.2 Matrix multiplication

```python
def matmul(A, B):
    """Matrix product by the definition (AB)_ij = sum_k A_ik B_kj.

    Three nested loops: O(m n p) operations.  We write it once to see what
    the @ operator does; afterwards we use @, which runs the same arithmetic
    in optimised compiled code.
    """
    A = _as_float_matrix(A)
    B = _as_float_matrix(B)
    m, n = A.shape
    n2, p = B.shape
    if n != n2:
        raise ValueError(f"inner dimensions differ: {A.shape} @ {B.shape}")
    C = np.zeros((m, p))
    for i in range(m):
        for j in range(p):
            s = 0.0
            for k in range(n):
                s += A[i, k] * B[k, j]
            C[i, j] = s
    return C


def outer(u, v):
    """Outer product u v^T: the (len(u) x len(v)) matrix with entries u_i v_j."""
    u = np.asarray(u, dtype=float).ravel()
    v = np.asarray(v, dtype=float).ravel()
    return u[:, None] * v[None, :]
```

demos/d01_matmul.py

```python
import time
import numpy as np
from linalg_scratch import matmul, outer

A = np.array([[1, 2], [0, 1]]); B = np.array([[3, 1], [1, 0]])  # Worked Ex. 2.2
print("matmul(A, B) =\n", matmul(A, B))
print("sum of outer products =\n",
      outer(A[:, 0], B[0, :]) + outer(A[:, 1], B[1, :]))

rng = np.random.default_rng(1)
X, Y = rng.standard_normal((120, 120)), rng.standard_normal((120, 120))
```

```
t0 = time.perf_counter(); C1 = matmul(X, Y); t1 = time.perf_counter()
C2 = X @ Y;                                  t2 = time.perf_counter()
print("max |difference| = %.2e" % np.max(np.abs(C1 - C2)))
print("triple loop: %.3f s,   @ operator: %.6f s" % (t1 - t0, t2 - t1))
```

OUTPUT

```
matmul(A, B) =
 [[5. 1.]
 [1. 0.]]
sum of outer products =
 [[5. 1.]
 [1. 0.]]
max |difference| = 1.78e-14
triple loop: 0.463 s,   @ operator: 0.000560 s
```

The triple loop gives the same answer as `@` up to rounding (10^{-14}, because the additions happen in a different order), but it is roughly a thousand times slower: Python executes each of the $120^3 \approx 1.7$ million multiply–adds one at a time, while `@` hands the whole job to optimised compiled code. From now on we use `@` — having written it ourselves, we know exactly what it does.

1.3 Gaussian elimination: rank and null space

Gauss–Jordan elimination brings a matrix to reduced row echelon form R. Two practical details matter:

- **Partial pivoting**: in each column we choose the entry of largest absolute value as the pivot. Dividing by a tiny pivot would create huge multipliers that amplify rounding errors.
- **A tolerance for "zero"**: after rounding, an entry that should be 0 might be 10^{-17}. We treat anything below $\text{tol} \cdot \max |a_{ij}|$ as zero. The choice of tolerance is a judgement call — which is one reason the SVD (Part 4) is the better tool for deciding rank.

```
def rref(A, tol=1e-10):
    """Reduced row echelon form by Gauss-Jordan elimination.

    Partial pivoting: in each column we pick the row with the largest
    |entry| as pivot, which keeps the multipliers <= 1 in size.
    Entries smaller than tol * max|A| count as zero.

    Returns (R, pivot_columns).
    """
    R = _as_float_matrix(A)
    m, n = R.shape
    scale = np.max(np.abs(R)) if R.size else 0.0
    thresh = tol * max(scale, 1.0)
    pivots = []
    row = 0
    for col in range(n):
        if row >= m:
```

```
            break
        # choose the pivot row: largest entry in this column, at or below `row`
        p = row + int(np.argmax(np.abs(R[row:, col])))
        if abs(R[p, col]) <= thresh:
            R[row:, col] = 0.0          # whole column is (numerically) zero
            continue
        R[[row, p]] = R[[p, row]]        # swap rows
        R[row] = R[row] / R[row, col]    # scale pivot to 1
        for i in range(m):               # eliminate above and below
            if i != row:
                R[i] = R[i] - R[i, col] * R[row]
        pivots.append(col)
        row += 1
    R[np.abs(R) <= thresh] = 0.0
    return R, pivots


def rank_by_elimination(A, tol=1e-10):
    """Rank = number of pivots in the reduced row echelon form."""
    return len(rref(A, tol)[1])


def null_space_by_elimination(A, tol=1e-10):
    """Basis of N(A): one 'special solution' per free variable (columns of the
    ↪ result)."""
    R, pivots = rref(A, tol)
    n = R.shape[1]
    free = [j for j in range(n) if j not in pivots]
    basis = []
    for f in free:
        x = np.zeros(n)
        x[f] = 1.0
        for i, pc in enumerate(pivots):  # pivot variable = -(its row's entry in column
    ↪ f)
            x[pc] = -R[i, f]
        basis.append(x)
    return np.array(basis).T if basis else np.zeros((n, 0))
```

demos/d01_rref.py

```
import numpy as np
from linalg_scratch import rref, rank_by_elimination, null_space_by_elimination

np.set_printoptions(precision=4, suppress=True)
A = np.array([[1, 2, 3], [2, 4, 7], [3, 6, 10]])        # Worked Example 2.3
R, pivots = rref(A)
print("R =\n", R)
print("pivot columns:", pivots, "  rank:", rank_by_elimination(A))
N = null_space_by_elimination(A)
print("null space basis (columns):\n", N)
print("left null space basis:\n", null_space_by_elimination(A.T))
print("A @ N =", (A @ N).ravel())

B = np.array([[1, 2, 0, 1], [2, 4, 1, 4], [3, 6, 1, 5]])  # Exercise 2.3
```

```
print("Exercise 2.3: rank", rank_by_elimination(B), " null space:\n",
      null_space_by_elimination(B))
```

OUTPUT

```
R =
 [[1. 2. 0.]
 [0. 0. 1.]
 [0. 0. 0.]]
pivot columns: [0, 2]   rank: 2
null space basis (columns):
 [[-2.]
 [ 1.]
 [-0.]]
left null space basis:
 [[-1.]
 [-1.]
 [ 1.]]
A @ N = [0. 0. 0.]
Exercise 2.3: rank 2  null space:
 [[-2. -1.]
 [ 1.  0.]
 [-0. -2.]
 [ 0.  1.]]
```

This is textbook Worked Example 2.3 and Exercise 2.3: rank 2, null space spanned by $(-2, 1, 0)$, left null space by $(1, 1, -1)$ up to sign, and the two special solutions $(-2, 1, 0, 0)$, $(-1, 0, -2, 1)$. (The `-0.` is a negative zero: harmless.)

Try it yourself.

1. Call `norm(np.array([1e-200, 1e-200]))` and compare with the naive $\sqrt{\sum x_i^2}$. What goes wrong now, and why does the scaling fix this case too?
2. Use `rref` on a 4×4 matrix whose last row is the sum of the first three, with 10^{-12} added to one entry. What rank do you get with `tol=1e-10` and with `tol=1e-14`? Which is "right"?

CHAPTER 2

Orthogonalisation and QR

Textbook: Chapter 3. **Assignment:** A2.

2.1 Two versions of Gram–Schmidt

The two functions below differ in a single expression: classical Gram–Schmidt computes $r_{ij} = \boldsymbol{q}_i \cdot \boldsymbol{a}_j$ with the *original* column; modified Gram–Schmidt computes $r_{ij} = \boldsymbol{q}_i \cdot \boldsymbol{w}$ with the *partially orthogonalised* vector. In exact arithmetic these are equal, because $\boldsymbol{w}$ differs from $\boldsymbol{a}_j$ only by multiples of $\boldsymbol{q}_1, \ldots, \boldsymbol{q}_{i-1}$, which are orthogonal to $\boldsymbol{q}_i$.

```
def gram_schmidt_classical(A):
    """Classical Gram-Schmidt: A = Q R with Q (m x n) orthonormal columns.

    r_ij = q_i . a_j uses the ORIGINAL column a_j.
    """
    A = _as_float_matrix(A)
    m, n = A.shape
    Q = np.zeros((m, n))
    R = np.zeros((n, n))
    for j in range(n):
        w = A[:, j].copy()
        for i in range(j):
            R[i, j] = Q[:, i] @ A[:, j]       # project the original a_j
            w -= R[i, j] * Q[:, i]
        R[j, j] = norm(w)
        if R[j, j] == 0.0:
            raise ValueError("columns are linearly dependent")
        Q[:, j] = w / R[j, j]
    return Q, R


def gram_schmidt_modified(A):
    """Modified Gram-Schmidt: same result in exact arithmetic.

    r_ij = q_i . w uses the PARTIALLY REDUCED vector w, so each subtraction
    also removes the rounding errors made by the previous ones.
    """
    A = _as_float_matrix(A)
    m, n = A.shape
    Q = np.zeros((m, n))
    R = np.zeros((n, n))
    for j in range(n):
        w = A[:, j].copy()
        for i in range(j):
            R[i, j] = Q[:, i] @ w             # project the current w
            w -= R[i, j] * Q[:, i]
```

```
        R[j, j] = norm(w)
        if R[j, j] == 0.0:
            raise ValueError("columns are linearly dependent")
        Q[:, j] = w / R[j, j]
    return Q, R
```

demos/d02_qr.py

```python
import numpy as np
from linalg_scratch import gram_schmidt_modified, householder_qr

np.set_printoptions(precision=4, suppress=True)
A = np.array([[1, 1, 0], [1, 0, 1], [0, 1, 1]])            # Worked Example 3.2
Q, R = gram_schmidt_modified(A)
print("Q =\n", Q)
print("R =\n", R)
print("sqrt(2), 1/sqrt(2), sqrt(3/2), 1/sqrt(6), 2/sqrt(3) =",
      np.round([np.sqrt(2), 1/np.sqrt(2), np.sqrt(1.5), 1/np.sqrt(6), 2/np.sqrt(3)], 4))
print("|Q^T Q - I| = %.1e,  |QR - A| = %.1e"
      % (np.abs(Q.T @ Q - np.eye(3)).max(), np.abs(Q @ R - A).max()))

Qh, Rh = householder_qr(A)
print("Householder R =\n", Rh)
print("(rows may differ from Gram-Schmidt by a sign: both are valid QRs)")
```

OUTPUT

```
Q =
 [[ 0.7071  0.4082 -0.5774]
 [ 0.7071 -0.4082  0.5774]
 [ 0.      0.8165  0.5774]]
R =
 [[1.4142 0.7071 0.7071]
 [0.     1.2247 0.4082]
 [0.     0.     1.1547]]
sqrt(2), 1/sqrt(2), sqrt(3/2), 1/sqrt(6), 2/sqrt(3) = [1.4142 0.7071 1.2247 0.4082 1.1547]
|Q^T Q - I| = 2.2e-16,  |QR - A| = 1.1e-16
Householder R =
 [[-1.4142 -0.7071 -0.7071]
 [ 0.      1.2247  0.4082]
 [ 0.      0.      1.1547]]
(rows may differ from Gram-Schmidt by a sign: both are valid QRs)
```

The output reproduces textbook Worked Examples 3.2 and 3.3: the entries of R are $\sqrt{2}$, $1/\sqrt{2}$, $\sqrt{3/2}$, $1/\sqrt{6}$ and $2/\sqrt{3}$.

2.2 Householder QR

Gram–Schmidt builds Q column by column and hopes it stays orthogonal. Householder's method instead builds R by applying *exactly orthogonal* reflections to A. Step k chooses a reflection $H_k = I - 2vv^{\mathsf{T}}/v^{\mathsf{T}}v$ that maps the part of column k on and below the diagonal to a multiple of e_1, zeroing everything below the diagonal. After n steps,

$$H_n \cdots H_2 H_1 A = R \quad \Longrightarrow \quad A = QR, \quad Q = H_1 H_2 \cdots H_n.$$

```
def householder_qr(A):
    """Full QR factorisation A = Q R by Householder reflections.

    Q is m x m orthogonal, R is m x n upper triangular.
    Step k reflects column k (from row k down) onto a multiple of e_1 with
    H = I - 2 v v^T / (v^T v),  v = x + sign(x_1) ||x|| e_1.
    The sign choice avoids subtracting two nearly equal numbers.
    """
    R = _as_float_matrix(A)
    m, n = R.shape
    Q = np.eye(m)
    for k in range(min(m - 1, n)):
        x = R[k:, k].copy()
        alpha = norm(x)
        if alpha == 0.0:
            continue                          # column already zero below the diagonal
        v = x.copy()
        v[0] += (1.0 if x[0] >= 0 else -1.0) * alpha
        vtv = v @ v
        # apply H = I - 2 v v^T / v^T v to the trailing rows of R ...
        R[k:, :] -= np.outer(v, (2.0 / vtv) * (v @ R[k:, :]))
        # ... and accumulate Q = H_1 H_2 ... H_k (apply H on the right)
        Q[:, k:] -= np.outer((Q[:, k:] @ v) * (2.0 / vtv), v)
        R[k + 1:, k] = 0.0                    # exact zeros below the diagonal
    return Q, R
```

Two implementation points. We never form the $m \times m$ matrix H_k: applying it to R costs only a matrix–vector product and an outer-product update, `R -= outer(v, (2/vtv) * (v @ R))`. And we add $\operatorname{sign}(x_1)\,\|x\|$ to x_1, never subtract, so that v_1 is never the difference of two nearly equal numbers. Householder's R may differ from Gram–Schmidt's by the signs of its rows (see the demo above); both are valid QR factorisations.

2.3 How much orthogonality survives?

The experiment below builds 40×20 matrices with prescribed condition number κ (singular values spread evenly on a log scale from 1 to $1/\kappa$), orthogonalises them three ways, and measures $\max \left| Q^{\mathsf{T}} Q - I \right|$.

demos/d02_stability.py

```python
import numpy as np
import matplotlib.pyplot as plt
from plotstyle import C1, C2, C3, savefig
from linalg_scratch import (householder_qr, gram_schmidt_classical,
                            gram_schmidt_modified)

def matrix_with_condition(kappa, m=40, n=20, seed=0):
    """A = U diag(s) V^T with singular values from 1 down to 1/kappa."""
    rng = np.random.default_rng(seed)
    U, _ = householder_qr(rng.standard_normal((m, m)))
    V, _ = householder_qr(rng.standard_normal((n, n)))
    s = np.logspace(0, -np.log10(kappa), n)
    return (U[:, :n] * s) @ V.T

def loss(Q):                        # how far Q is from orthonormal columns
    k = Q.shape[1]
    return np.abs(Q.T @ Q - np.eye(k)).max()

kappas = np.logspace(1, 15, 15)
res = {"classical GS": [], "modified GS": [], "Householder": []}
for kap in kappas:
    A = matrix_with_condition(kap)
    res["classical GS"].append(loss(gram_schmidt_classical(A)[0]))
    res["modified GS"].append(loss(gram_schmidt_modified(A)[0]))
    res["Householder"].append(loss(householder_qr(A)[0][:, :A.shape[1]]))

print(" kappa     classical    modified   Householder")
for i in range(0, 15, 2):
    print("%7.0e   %9.1e   %9.1e   %9.1e" % (kappas[i], res["classical GS"][i],
          res["modified GS"][i], res["Householder"][i]))

for (lab, val), c, mk in zip(res.items(), (C1, C2, C3), ("o", "s", "^")):
    plt.loglog(kappas, val, marker=mk, color=c, label=lab)
plt.xlabel("condition number $\\kappa(A)$")
plt.ylabel("$\\max|Q^TQ-I|$")
plt.legend(); savefig("gs_stability")
```

OUTPUT

```
 kappa     classical    modified   Householder
  1e+01     4.4e-16     4.4e-16     8.9e-16
  1e+03     2.8e-12     1.1e-14     7.8e-16
  1e+05     1.1e-08     2.9e-12     8.9e-16
  1e+07     2.8e-04     5.1e-11     8.9e-16
  1e+09     9.8e-01     1.2e-08     8.9e-16
  1e+11     1.0e+00     3.5e-06     6.7e-16
  1e+13     1.0e+00     1.6e-04     1.6e-15
```

```
1e+15      1.0e+00      2.8e-02      8.9e-16
```

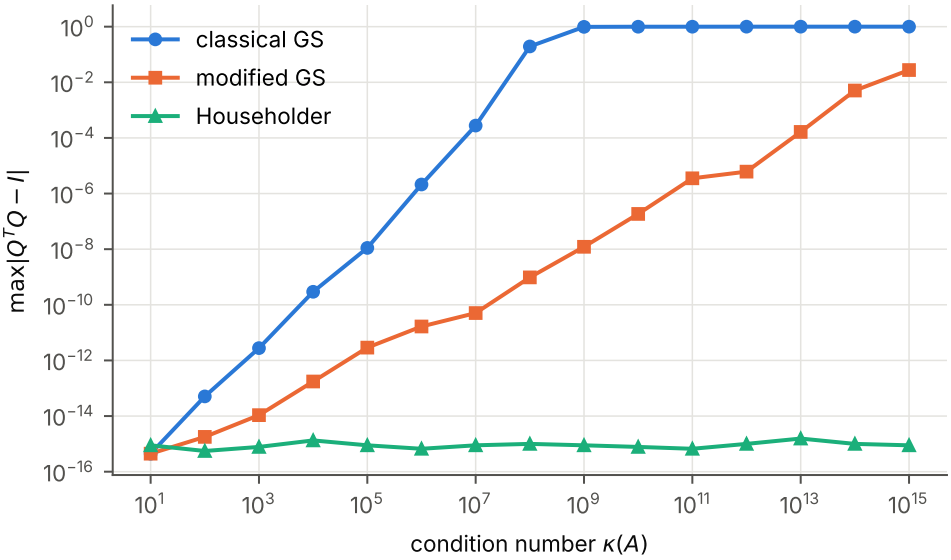


Figure 2.1: Loss of orthogonality. Classical Gram–Schmidt degrades like $\kappa^2\varepsilon$ and is useless by $\kappa \approx 10^8$; modified Gram–Schmidt degrades like $\kappa\varepsilon$; Householder stays orthogonal to machine precision regardless of κ.

This is the decisive reason libraries use Householder reflections. Modified Gram–Schmidt is still useful: it produces the reduced Q directly and is easier to update when columns are added.

2.4 Triangular systems, solving, projections

With QR in hand, solving a square system $A\boldsymbol{x} = \boldsymbol{b}$ is two steps: $Q^{\mathsf{T}}\boldsymbol{b}$, then back substitution with R. We also need forward substitution (for Cholesky, Part 6), the projection matrix QQ^{T}, and a routine that extends orthonormal columns to a full orthogonal matrix — the SVD needs it to fill in the columns of U that belong to the left null space (textbook, recipe step 5).

```python
def back_substitution(U, b):
    """Solve U x = b for upper-triangular U (b may be a vector or a matrix)."""
    U = _as_float_matrix(U)
    b = np.array(b, dtype=float)
    n = U.shape[0]
    x = np.zeros_like(b, dtype=float)
    for i in range(n - 1, -1, -1):
        if U[i, i] == 0.0:
            raise ValueError("singular triangular matrix")
        x[i] = (b[i] - U[i, i + 1:] @ x[i + 1:]) / U[i, i]
    return x


def forward_substitution(L, b):
    """Solve L x = b for lower-triangular L."""
    L = _as_float_matrix(L)
    b = np.array(b, dtype=float)
    n = L.shape[0]
```

```
    x = np.zeros_like(b, dtype=float)
    for i in range(n):
        if L[i, i] == 0.0:
            raise ValueError("singular triangular matrix")
        x[i] = (b[i] - L[i, :i] @ x[:i]) / L[i, i]
    return x


def solve(A, b):
    """Solve a square system A x = b via Householder QR: R x = Q^T b."""
    A = _as_float_matrix(A)
    if A.shape[0] != A.shape[1]:
        raise ValueError("solve() needs a square matrix")
    Q, R = householder_qr(A)
    return back_substitution(R, Q.T @ np.asarray(b, dtype=float))
```

```
def projection_matrix(A):
    """Orthogonal projector onto C(A) for A with independent columns: P = Q Q^T."""
    A = _as_float_matrix(A)
    n = A.shape[1]
    Q, _ = householder_qr(A)
    Qn = Q[:, :n]                # first n columns: orthonormal basis of C(A)
    return Qn @ Qn.T


def complete_orthonormal_basis(Q, tol=1e-10):
    """Extend the orthonormal columns of Q (m x k) to an m x m orthogonal matrix.

    Try the standard basis vectors e_1, e_2, ... in turn; remove their
    components along the columns we already have (twice, for safety --
    'twice is enough'); keep the result if anything substantial is left.
    """
    Q = _as_float_matrix(Q)
    m, k = Q.shape
    cols = [Q[:, j] for j in range(k)]
    for i in range(m):
        if len(cols) == m:
            break
        w = np.zeros(m)
        w[i] = 1.0
        for _ in range(2):
            for q in cols:
                w -= (q @ w) * q
        nw = norm(w)
        if nw > tol:
            cols.append(w / nw)
    return np.column_stack(cols) if cols else np.zeros((m, 0))
```

demos/d02_projection.py

```python
import numpy as np
from linalg_scratch import projection_matrix, complete_orthonormal_basis

np.set_printoptions(precision=4, suppress=True)
A = np.array([[1, 0], [1, 1], [1, 2]]); b = np.array([6, 0, 0])   # Worked Ex. 3.1
P = projection_matrix(A)
print("6 P =\n", 6 * P)
print("P b =", P @ b, "   error e =", b - P @ b)
print("|P@P - P| = %.1e,  |P - P^T| = %.1e"
      % (np.abs(P @ P - P).max(), np.abs(P - P.T).max()))

q = np.array([[1, 1, 1]]).T / np.sqrt(3)                # extend one unit vector
Q = complete_orthonormal_basis(q)
print("completed basis Q =\n", Q)
print("|Q^T Q - I| = %.1e" % np.abs(Q.T @ Q - np.eye(3)).max())
```

OUTPUT

```
6 P =
 [[ 5.  2. -1.]
 [ 2.  2.  2.]
 [-1.  2.  5.]]
P b = [ 5.  2. -1.]    error e = [ 1. -2.  1.]
|P@P - P| = 4.4e-16,  |P - P^T| = 0.0e+00
completed basis Q =
 [[ 0.5774  0.8165 -0.    ]
 [ 0.5774 -0.4082  0.7071]
 [ 0.5774 -0.4082 -0.7071]]
|Q^T Q - I| = 2.2e-16
```

The first part is textbook Worked Example 3.1: $6P$ has integer entries, $P\boldsymbol{b} = (5, 2, -1)$, and the error is $(1, -2, 1)$. In `complete_orthonormal_basis` we orthogonalise each candidate *twice*: one pass of Gram–Schmidt can leave a residual component of size ε times the cancelled amount, and a second pass removes it ("twice is enough").

Try it yourself.

1. Repeat the stability experiment with `gram_schmidt_classical` applied twice in a row (orthogonalise A, then orthogonalise the resulting Q). How close does "classical GS twice" come to Householder?
2. Write `householder_qr_reduced(A)` returning the $m \times n$ matrix Q and the $n \times n$ matrix R. Check $QR = A$ on a 100×10 random matrix.

CHAPTER 3

Eigenvalues of Symmetric Matrices

Textbook: Chapters 4–5. **Assignment:** A3.

3.1 Power iteration and deflation

```python
def rayleigh_quotient(S, x):
    """R(x) = x^T S x / x^T x."""
    x = np.asarray(x, dtype=float)
    return float(x @ (S @ x) / (x @ x))


def power_iteration(S, x0=None, tol=1e-12, max_iter=10_000, seed=0):
    """Dominant eigenpair of a symmetric matrix S.

    Returns (lam, v, history) where history[k] is the Rayleigh quotient
    after k+1 steps.  Converges when |lambda_1| > |lambda_2|.
    """
    S = _as_float_matrix(S)
    n = S.shape[0]
    if x0 is None:
        x0 = np.random.default_rng(seed).standard_normal(n)
    x = normalize(x0)
    lam = rayleigh_quotient(S, x)
    history = []
    for _ in range(max_iter):
        y = S @ x
        ny = norm(y)
        if ny == 0.0:                # x is in the null space: eigenvalue 0
            return 0.0, x, history + [0.0]
        x = y / ny
        lam_new = rayleigh_quotient(S, x)
        history.append(lam_new)
        if abs(lam_new - lam) <= tol * max(1.0, abs(lam_new)):
            lam = lam_new
            break
        lam = lam_new
    return lam, x, history


def eigs_by_deflation(S, k, **kwargs):
    """Top-k eigenpairs of symmetric PSD S: power iteration + deflation.

    After finding (lam, v) we replace S by S - lam v v^T, which moves that
    eigenvalue to 0 and leaves the others unchanged.
    """
    S = _as_float_matrix(S).copy()
```

```
    lams, vecs = [], []
    for _ in range(k):
        lam, v, _ = power_iteration(S, **kwargs)
        lams.append(lam)
        vecs.append(v)
        S = S - lam * np.outer(v, v)
    return np.array(lams), np.column_stack(vecs)
```

demos/d03_power.py

```
import numpy as np
import matplotlib.pyplot as plt
from plotstyle import C1, C2, C3, savefig
from linalg_scratch import power_iteration, rayleigh_quotient

S = np.array([[2.0, 1.0], [1.0, 2.0]])                  # Worked Example 4.4
x = np.array([1.0, 0.0])
for k in range(4):
    print("k=%d  x=%-12s  R(x)=%.6f  error=%.6f"
          % (k, x.astype(int), rayleigh_quotient(S, x), 3 - rayleigh_quotient(S, x)))
    x = S @ x

lam, v, hist = power_iteration(S, x0=[1.0, 0.0])
print("converged: lambda = %.12f, v = %s, steps = %d" % (lam, v, len(hist)))

# convergence speed depends on the ratio lambda_2 / lambda_1
for ratio, c in zip((0.3, 0.7, 0.9), (C1, C2, C3)):
    D = np.diag([1.0, ratio, ratio / 2])
    _, _, h = power_iteration(D, x0=[1.0, 1.0, 1.0], tol=1e-16, max_iter=60)
    plt.semilogy(range(1, len(h) + 1), np.abs(1.0 - np.array(h)) + 1e-17,
                 color=c, label="$\\lambda_2/\\lambda_1=%.1f$" % ratio)
plt.ylim(1e-16, 1); plt.xlabel("iteration $k$"); plt.ylabel("error in $\\lambda_1$")
plt.legend(); savefig("power_convergence")
```

OUTPUT

```
k=0  x=[1 0]         R(x)=2.000000  error=1.000000
k=1  x=[2 1]         R(x)=2.800000  error=0.200000
k=2  x=[5 4]         R(x)=2.975610  error=0.024390
k=3  x=[14 13]       R(x)=2.997260  error=0.002740
converged: lambda = 3.000000000000, v = [0.70710693 0.70710663], steps = 14
```

The table reproduces textbook Worked Example 4.4 (the Rayleigh quotients 2, 2.8, 2.9756, 2.9973). Run to convergence, the method finds $\lambda = 3$ and $v \approx \frac{1}{\sqrt{2}}(1,1)$ in 14 steps. Notice that v is correct only to about 7 digits while λ is correct to 12: the Rayleigh quotient's error is the *square* of the vector's error, as the textbook explains.

Power iteration only finds the dominant eigenvalue, it converges slowly when $|\lambda_2| \approx |\lambda_1|$, and deflation accumulates errors from one eigenpair into the next. We need something better for a complete decomposition.

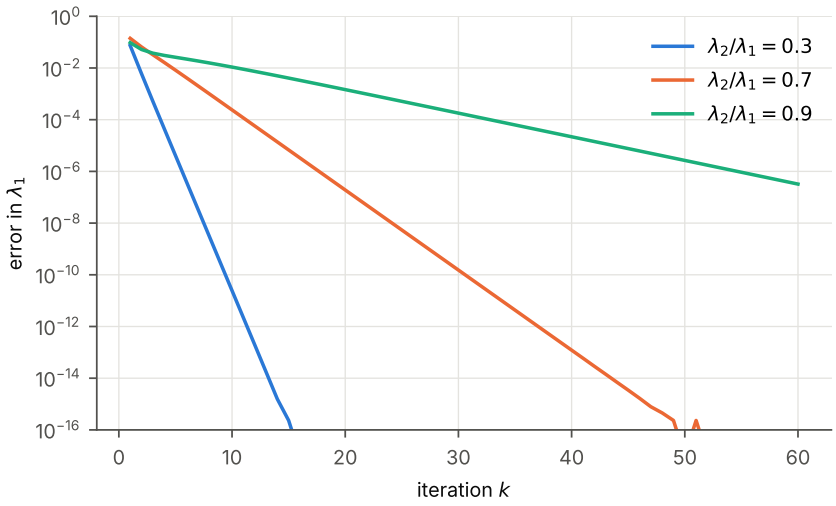


Figure 3.1: Error in λ_1 for power iteration on $\mathrm{diag}(1, r, r/2)$. The error falls by a factor r^2 per step: fast when the eigenvalues are well separated, slow when λ_2/λ_1 is close to 1.

3.2 The Jacobi eigenvalue algorithm

Jacobi's method (1846) is the simplest algorithm that computes *all* eigenvalues and eigenvectors of a symmetric matrix accurately. Pick an off-diagonal pair (p, q) and a rotation J in the (p, q) plane,

$$J = \begin{bmatrix} c & s \\ -s & c \end{bmatrix} \text{ (in rows/columns } p, q), \qquad c = \cos\theta,\ s = \sin\theta,$$

chosen so that $S' = J^{\mathsf{T}} S J$ has $s'_{pq} = 0$. Working out the 2×2 block gives the condition

$$\frac{c^2 - s^2}{cs} = \frac{s_{qq} - s_{pp}}{s_{pq}} = 2\vartheta, \qquad \text{i.e.} \quad t^2 + 2\vartheta t - 1 = 0 \ \text{ for } t = \tan\theta.$$

We take the smaller root $t = \mathrm{sign}(\vartheta)/(|\vartheta| + \sqrt{\vartheta^2 + 1})$, so that $|\theta| \le \pi/4$ and the rotation disturbs the matrix as little as possible. Each rotation moves the "mass" $2s_{pq}^2$ from the off-diagonal part onto the diagonal, so the off-diagonal norm decreases steadily; sweeping over all pairs repeatedly drives it to zero. The diagonal then holds the eigenvalues, and the product of all the rotations holds the eigenvectors.

```
391 def jacobi_eigh(S, tol=1e-14, max_sweeps=60, return_history=False):
392     """All eigenvalues and eigenvectors of a symmetric matrix (cyclic Jacobi).
393 
394     Each step picks an off-diagonal entry s_pq and applies a plane rotation
395     J (in the p,q plane) with  S <- J^T S J  chosen so that the new s_pq = 0.
396     One 'sweep' visits every pair p < q once.  The off-diagonal mass
397     off(S) = sqrt(sum_{i != j} s_ij^2) decreases to zero (quadratically
398     near the end), and the product of all rotations holds the eigenvectors.
399 
400     Returns (eigenvalues sorted in DECREASING order, V with eigenvectors as
401     columns) -- and the list of off(S) after each sweep if requested.
402     """
403     A = _as_float_matrix(S).copy()
```

```
    n = A.shape[0]
    if A.shape != (n, n):
        raise ValueError("matrix must be square")
    if not np.allclose(A, A.T, atol=1e-12 * max(1.0, np.max(np.abs(A)))):
        raise ValueError("matrix must be symmetric")
    V = np.eye(n)
    history = []

    def off(M):
        # sum the off-diagonal squares directly; computing sum(M*M) - sum(diag^2)
        # would cancel catastrophically once the off-diagonal part is tiny
        O = M - np.diag(np.diag(M))
        return np.sqrt(np.sum(O * O))

    scale = np.sqrt(np.sum(A * A)) or 1.0
    for _ in range(max_sweeps):
        history.append(off(A))
        if off(A) <= tol * scale:
            break
        for p in range(n - 1):
            for q in range(p + 1, n):
                apq = A[p, q]
                if abs(apq) <= 1e-300:
                    continue
                # rotation angle: t = tan(theta) solves t^2 + 2 t theta_ - 1 = 0
                theta = (A[q, q] - A[p, p]) / (2.0 * apq)
                if abs(theta) > 1e150:              # avoid overflow in theta^2
                    t = 0.5 / theta
                else:
                    t = (1.0 if theta >= 0 else -1.0) / (abs(theta) + np.sqrt(theta *
    ↪ theta + 1.0))
                c = 1.0 / np.sqrt(t * t + 1.0)
                s = t * c
                # A <- A J  (columns p and q)
                Ap, Aq = A[:, p].copy(), A[:, q].copy()
                A[:, p] = c * Ap - s * Aq
                A[:, q] = s * Ap + c * Aq
                # A <- J^T A  (rows p and q)
                Ap, Aq = A[p, :].copy(), A[q, :].copy()
                A[p, :] = c * Ap - s * Aq
                A[q, :] = s * Ap + c * Aq
                A[p, q] = A[q, p] = 0.0        # exactly zero by construction
                # accumulate eigenvectors V <- V J
                Vp, Vq = V[:, p].copy(), V[:, q].copy()
                V[:, p] = c * Vp - s * Vq
                V[:, q] = s * Vp + c * Vq
    else:
        history.append(off(A))
    eigvals = np.diag(A).copy()
    order = np.argsort(eigvals)[::-1]          # largest first
    eigvals, V = eigvals[order], V[:, order]
    return (eigvals, V, history) if return_history else (eigvals, V)
```

demos/d03_jacobi.py

```python
import numpy as np
from linalg_scratch import jacobi_eigh

np.set_printoptions(precision=4, suppress=True)
S = np.array([[2, 1, 1], [1, 2, 1], [1, 1, 2]])           # Worked Example 4.3
lam, V = jacobi_eigh(S)
print("eigenvalues:", lam)
print("eigenvectors (columns):\n", V)
print("|S V - V diag(lam)| = %.1e" % np.abs(S @ V - V * lam).max())

rng = np.random.default_rng(3)
B = rng.standard_normal((8, 8)); S8 = B + B.T             # random symmetric 8 x 8
lam8, V8, hist = jacobi_eigh(S8, return_history=True)
print("off-diagonal norm after each sweep:")
print("  " + "  ".join("%.1e" % h for h in hist))
ref = np.sort(np.linalg.eigvalsh(S8))[::-1]               # NumPy, for checking only
print("max |our eigenvalues - NumPy's| = %.1e" % np.abs(lam8 - ref).max())
```

OUTPUT

```
eigenvalues: [4. 1. 1.]
eigenvectors (columns):
 [[ 0.5774 -0.4082  0.7071]
 [ 0.5774 -0.4082 -0.7071]
 [ 0.5774  0.8165  0.    ]]
|S V - V diag(lam)| = 4.4e-16
off-diagonal norm after each sweep:
  1.0e+01  4.8e+00  1.0e+00  6.4e-02  6.3e-05  6.9e-11  7.4e-29
max |our eigenvalues - NumPy's| = 3.6e-15
```

On textbook Worked Example 4.3 Jacobi finds $\lambda = 4, 1, 1$; the two eigenvectors for the repeated eigenvalue 1 differ from the ones in the textbook, as they are free to (any orthonormal basis of the plane $x + y + z = 0$ will do). On a random 8×8 matrix, look at how the off-diagonal norm falls: 6×10^{-5}, then 7×10^{-11}, then 10^{-28}. The number of correct digits roughly doubles with every sweep near the end — *quadratic convergence*.

A bug we made while writing this manual. Our first version computed the off-diagonal norm as `sqrt(sum(M*M) - sum(diag(M)**2))`. Once the off-diagonal part is around 10^{-7} of the whole, its square is around 10^{-14} of the whole, and the subtraction cancels catastrophically to exactly 0. The loop then stopped early, and the smallest eigenvalues came out with only a few correct digits. The fixed version sums the off-diagonal squares directly. Lesson: *never compute a small quantity as the difference of two large ones* (Part 0, lesson 3).

Try it yourself.

1. Use `eigs_by_deflation` to find the top three eigenpairs of a random 6×6 PSD matrix $B^{\mathsf{T}}B$, and compare with `jacobi_eigh`. How does the accuracy of the third eigenvalue compare to the first?
2. Count the rotations Jacobi needs for $n = 10, 20, 40$. How does the work grow with n?

CHAPTER 4

The Singular Value Decomposition

Textbook: Chapters 6–7. **Assignment:** A4.

We implement two SVD algorithms. Both return `(U, s, Vt)` with the same conventions as `numpy.linalg.svd`: `s` is a 1-D array in decreasing order and `Vt` is V^T (rows are the right singular vectors). With `full_matrices=False` we return the reduced SVD (U is $m \times k$, V^T is $k \times n$, $k = \min(m, n)$); with `True`, the full one.

4.1 Method 1: the textbook recipe

This is exactly the hand method of textbook Chapter 6: eigen-decompose $A^\mathsf{T}A$, take square roots, set $\boldsymbol{u}_i = A\boldsymbol{v}_i/\sigma_i$, and complete U.

```
def svd_via_eig(A, full_matrices=False):
    """SVD from the eigen-decomposition of A^T A (the textbook hand method).

    1. eigenpairs of the smaller Gram matrix (A^T A, or A A^T for wide A)
    2. sigma_i = sqrt(lambda_i)  (clip tiny negative rounding to 0)
    3. u_i = A v_i / sigma_i     (never compute U independently!)
    4. complete U (and V) with orthonormal bases of the null spaces.

    Returns U, s, Vt like numpy.linalg.svd.  Simple, but it squares the
    condition number: singular values below ~ sqrt(eps) * sigma_1 are lost
    (they are returned as 0).
    """
    A = _as_float_matrix(A)
    m, n = A.shape
    if m < n:                                   # wide: work with A^T and swap back
        U, s, Vt = svd_via_eig(A.T, full_matrices)
        return Vt.T, s, U.T
    k = n                                       # now m >= n, so min(m, n) = n
    lam, V = jacobi_eigh(A.T @ A)
    s = np.sqrt(np.clip(lam, 0.0, None))
    # rank decision on the EIGENVALUES: lambda <= max(m,n) eps lambda_1 is noise,
    # i.e. this method cannot resolve sigma below about sqrt(eps) * sigma_1
    tol = max(m, n) * EPS * (lam[0] if lam.size and lam[0] > 0 else 1.0)
    r = int(np.sum(lam > tol))                  # numerical rank
    U = np.zeros((m, r))
    for i in range(r):
        U[:, i] = (A @ V[:, i]) / s[i]
    U = complete_orthonormal_basis(U)           # m x m orthogonal
    s[r:] = 0.0
    if full_matrices:
        return U, s, V.T
    return U[:, :k], s, V.T
```

Three details: wide matrices are handled by decomposing A^{T} and swapping U and V; tiny negative eigenvalues (rounding) are clipped to 0 before the square root; and the rank is decided on the *eigenvalues*: any $\lambda_i \le \max(m,n)\,\varepsilon\,\lambda_1$ is rounding noise, so its σ_i is set to exactly 0 and we never divide by noise when forming $\boldsymbol{u}_i$. Note what this means: the method cannot see singular values below about $\sqrt{\varepsilon}\,\sigma_1 \approx 10^{-8}\sigma_1$ at all.

4.2 Method 2: one-sided Jacobi

The one-sided Jacobi method (Hestenes, 1958) never forms $A^{\mathsf{T}}A$. It applies rotations to the *columns* of a working copy $W = AV$, choosing each rotation to make one pair of columns orthogonal. When all columns of W are mutually orthogonal, $W = U\Sigma$: the column lengths are the singular values and the normalised columns are the left singular vectors. The rotation for columns p, q is the Jacobi rotation of Part 3 applied to the 2×2 Gram matrix $\left[\begin{smallmatrix} \alpha & \gamma \\ \gamma & \beta \end{smallmatrix}\right]$ with $\alpha = \|\boldsymbol{w}_p\|^2$, $\beta = \|\boldsymbol{w}_q\|^2$, $\gamma = \boldsymbol{w}_p \cdot \boldsymbol{w}_q$ — so it is mathematically the same as running Jacobi on $A^{\mathsf{T}}A$, but it only ever touches A.

```
def svd_jacobi(A, full_matrices=False, tol=1e-15, max_sweeps=60):
    """SVD by one-sided Jacobi (Hestenes' method).

    Rotate PAIRS OF COLUMNS of a working copy W = A V until all columns of W
    are mutually orthogonal.  Then W = U Sigma: the column norms are the
    singular values and the normalised columns are the u_i.  A^T A is never
    formed, so small singular values are computed to high relative accuracy.

    For columns w_p, w_q let  alpha = |w_p|^2, beta = |w_q|^2, gamma = w_p.w_q.
    The rotation that makes them orthogonal is the Jacobi rotation that
    diagonalises the 2 x 2 matrix [[alpha, gamma], [gamma, beta]].
    """
    A = _as_float_matrix(A)
    m, n = A.shape
    if m < n:                                   # wide: decompose A^T and swap back
        U, s, Vt = svd_jacobi(A.T, full_matrices, tol, max_sweeps)
        return Vt.T, s, U.T
    W = A.copy()
    V = np.eye(n)
    for _ in range(max_sweeps):
        rotated = False
        for p in range(n - 1):
            for q in range(p + 1, n):
                alpha = W[:, p] @ W[:, p]
                beta = W[:, q] @ W[:, q]
                gamma = W[:, p] @ W[:, q]
                if abs(gamma) <= tol * np.sqrt(alpha * beta) or gamma == 0.0:
                    continue                    # already orthogonal (to working
    ↪ accuracy)
                rotated = True
                zeta = (beta - alpha) / (2.0 * gamma)
                if abs(zeta) > 1e150:               # avoid overflow in zeta^2
                    t = 0.5 / zeta
                else:
                    t = (1.0 if zeta >= 0 else -1.0) / (abs(zeta) + np.sqrt(1.0 + zeta *
```

```
    ↪ zeta))
                c = 1.0 / np.sqrt(1.0 + t * t)
                s_ = c * t
                Wp, Wq = W[:, p].copy(), W[:, q].copy()
                W[:, p] = c * Wp - s_ * Wq
                W[:, q] = s_ * Wp + c * Wq
                Vp, Vq = V[:, p].copy(), V[:, q].copy()
                V[:, p] = c * Vp - s_ * Vq
                V[:, q] = s_ * Vp + c * Vq
        if not rotated:
            break
    s = np.array([norm(W[:, j]) for j in range(n)])
    order = np.argsort(s)[::-1]                # sort singular values, largest first
    s, W, V = s[order], W[:, order], V[:, order]
    tol_rank = max(m, n) * EPS * (s[0] if s.size and s[0] > 0 else 1.0)
    r = int(np.sum(s > tol_rank))
    U = np.zeros((m, r))
    for i in range(r):
        U[:, i] = W[:, i] / s[i]
    U = complete_orthonormal_basis(U)          # fill in u_{r+1}, ..., u_m
    s[r:] = 0.0
    if full_matrices:
        return U, s, V.T
    return U[:, :n], s, V.T


def svd(A, full_matrices=False):
    """Our default SVD: one-sided Jacobi."""
    return svd_jacobi(A, full_matrices=full_matrices)
```

4.3 Checking against the worked examples

demos/d04_svd_examples.py

```
import numpy as np
from linalg_scratch import svd_via_eig, svd_jacobi

np.set_printoptions(precision=4, suppress=True)
examples = {"Worked Ex. 6.1": [[3, 0], [4, 5]],
            "Worked Ex. 6.2": [[3, 2, 2], [2, 3, -2]],
            "Worked Ex. 6.3": [[1, 1], [1, 1], [1, 1]],
            "Worked Ex. 6.4": [[1, 1], [0, 1], [1, 0]]}
for name, A in examples.items():
    A = np.array(A, dtype=float)
    U, s, Vt = svd_jacobi(A, full_matrices=True)
    Sigma = np.zeros(A.shape); Sigma[:s.size, :s.size] = np.diag(s)
    print(f"--- {name}:  sigma = {s}")
    print("U =\n", U); print("V^T =\n", Vt)
    print("|U Sigma V^T - A| = %.1e   |U^T U - I| = %.1e   |V^T V - I| = %.1e"
          % (np.abs(U @ Sigma @ Vt - A).max(), np.abs(U.T @ U -
    ↪ np.eye(U.shape[0])).max(),
             np.abs(Vt @ Vt.T - np.eye(Vt.shape[0])).max()))
    s2 = svd_via_eig(A)[1]
```

```
    print("svd_via_eig sigma =", s2, "  NumPy:", np.linalg.svd(A, compute_uv=False))
```

OUTPUT

```
--- Worked Ex. 6.1:  sigma = [6.7082 2.2361]
U =
 [[ 0.3162  0.9487]
 [ 0.9487 -0.3162]]
V^T =
 [[ 0.7071  0.7071]
 [ 0.7071 -0.7071]]
|U Sigma V^T - A| = 8.9e-16   |U^T U - I| = 3.3e-16   |V^T V - I| = 2.2e-16
svd_via_eig sigma = [6.7082 2.2361]   NumPy: [6.7082 2.2361]
--- Worked Ex. 6.2:  sigma = [5. 3.]
U =
 [[ 0.7071  0.7071]
 [ 0.7071 -0.7071]]
V^T =
 [[ 0.7071  0.7071  0.    ]
 [ 0.2357 -0.2357  0.9428]
 [ 0.6667 -0.6667 -0.3333]]
|U Sigma V^T - A| = 4.4e-16   |U^T U - I| = 2.2e-16   |V^T V - I| = 2.2e-16
svd_via_eig sigma = [5. 3.]   NumPy: [5. 3.]
--- Worked Ex. 6.3:  sigma = [2.4495 0.    ]
U =
 [[ 0.5774  0.8165 -0.    ]
 [ 0.5774 -0.4082  0.7071]
 [ 0.5774 -0.4082 -0.7071]]
V^T =
 [[ 0.7071  0.7071]
 [ 0.7071 -0.7071]]
|U Sigma V^T - A| = 2.2e-16   |U^T U - I| = 2.2e-16   |V^T V - I| = 2.2e-16
svd_via_eig sigma = [2.4495 0.    ]   NumPy: [2.4495 0.    ]
--- Worked Ex. 6.4:  sigma = [1.7321 1.    ]
U =
 [[ 0.8165  0.      0.5774]
 [ 0.4082 -0.7071 -0.5774]
 [ 0.4082  0.7071 -0.5774]]
V^T =
 [[ 0.7071  0.7071]
 [ 0.7071 -0.7071]]
|U Sigma V^T - A| = 2.2e-16   |U^T U - I| = 2.2e-16   |V^T V - I| = 2.2e-16
svd_via_eig sigma = [1.7321 1.    ]   NumPy: [1.7321 1.    ]
```

All four worked examples of textbook Chapter 6 are reproduced: $\sigma = (3\sqrt{5}, \sqrt{5}) \approx (6.7082, 2.2361)$; $(5, 3)$; $(\sqrt{6}, 0)$; $(\sqrt{3}, 1)$. The singular vectors match the hand calculations up to sign — for example, in Worked Example 6.2 the third column of V is $\frac{1}{3}(2, -2, -1)$ and $v_2 = \frac{1}{3\sqrt{2}}(1, -1, 4) \approx (0.2357, -0.2357, 0.9428)$. For the rank-one matrix, the last two columns of U are a valid but different orthonormal basis of $\mathcal{N}(A^{\mathsf{T}})$ than the textbook's, which is allowed. The reconstruction error $\|U\Sigma V^{\mathsf{T}} - A\|$ and the orthogonality errors are all at the level of $\varepsilon_{\text{mach}}$.

4.4 Accuracy: why libraries don't form $A^{\mathsf{T}}A$

The experiment below first tries both methods on the Läuchli matrix of textbook Worked Example 10.5, and then on matrices of growing condition number.

demos/d04_accuracy.py

```python
import numpy as np
import matplotlib.pyplot as plt
from plotstyle import C1, C2, C3, savefig
from linalg_scratch import svd_via_eig, svd_jacobi

eps = 1e-8
L = np.array([[1, 1], [eps, 0], [0, eps]])              # the Lauchli matrix
print("true sigma_2      = 1.0000e-08")
print("svd_via_eig  gives %.4e" % svd_via_eig(L)[1][1])
print("svd_jacobi   gives %.4e" % svd_jacobi(L)[1][1])

# relative error of the smallest singular value as kappa grows
rng = np.random.default_rng(0)
Q1 = np.linalg.qr(rng.standard_normal((30, 30)))[0]     # NumPy only builds the
Q2 = np.linalg.qr(rng.standard_normal((10, 10)))[0]     # test matrices here
kappas = np.logspace(1, 14, 14)
err_eig, err_jac = [], []
for kap in kappas:
    s = np.logspace(0, -np.log10(kap), 10)
    A = (Q1[:, :10] * s) @ Q2.T
    err_eig.append(abs(svd_via_eig(A)[1][-1] - s[-1]) / s[-1])
    err_jac.append(abs(svd_jacobi(A)[1][-1] - s[-1]) / s[-1])
plt.loglog(kappas, np.maximum(err_eig, 1e-17), "o-", color=C1, label="via eig$(A^TA)$")
plt.loglog(kappas, np.maximum(err_jac, 1e-17), "s-", color=C2, label="one-sided Jacobi")
eps_m = np.finfo(float).eps
plt.loglog(kappas, eps_m * kappas, color="#52514e", lw=0.8, ls=":")
plt.loglog(kappas, np.minimum(eps_m * kappas ** 2, 1), color="#52514e", lw=0.8, ls="--")
plt.text(3e11, 4e-5, "$\\kappa\\varepsilon$", color="#52514e")
plt.text(2e6, 1e-3, "$\\kappa^2\\varepsilon$", color="#52514e")
plt.xlabel("condition number $\\kappa(A)$")
plt.ylabel("relative error in $\\sigma_{\\min}$"); plt.legend(); savefig("svd_accuracy")
for k, e1, e2 in list(zip(kappas, err_eig, err_jac))[::3]:
    print("kappa=%.0e   eig-based error %.1e   Jacobi error %.1e" % (k, e1, e2))
```

OUTPUT

```
true sigma_2      = 1.0000e-08
svd_via_eig  gives 0.0000e+00
svd_jacobi   gives 1.0000e-08
kappa=1e+01   eig-based error 2.8e-16   Jacobi error 6.9e-16
kappa=1e+04   eig-based error 6.8e-10   Jacobi error 1.3e-14
kappa=1e+07   eig-based error 8.2e-05   Jacobi error 4.8e-11
kappa=1e+10   eig-based error 1.0e+00   Jacobi error 6.3e-08
kappa=1e+13   eig-based error 1.0e+00   Jacobi error 3.4e-05
```

The Läuchli matrix of textbook Worked Example 10.5 is the extreme case: the eigenvalue route returns $\sigma_2 = 0$ while Jacobi returns 10^{-8}, correct to all digits shown.

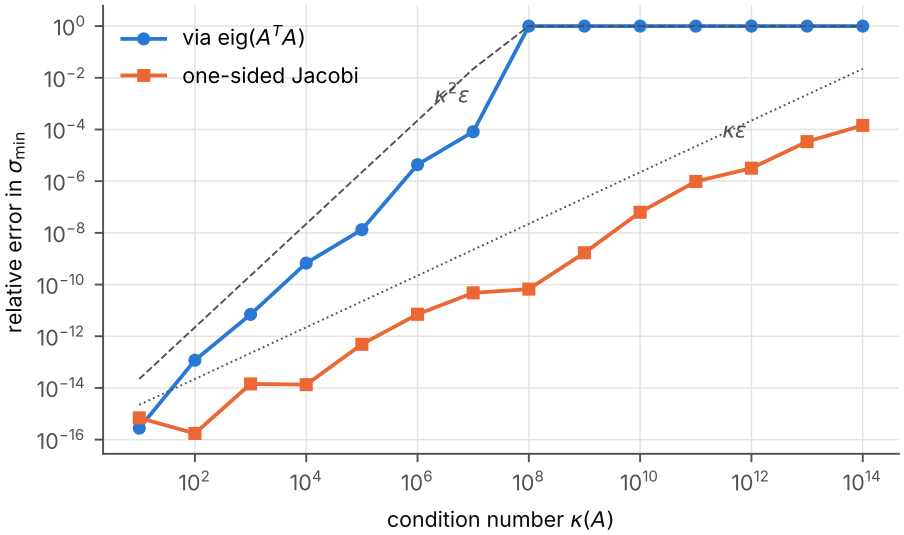


Figure 4.1: Relative error in the smallest singular value. Forming $A^{\mathsf{T}}A$ (blue) gives errors around $\kappa^2\varepsilon$ and loses $\sigma_{\min}$ completely once $\kappa \gtrsim 10^8$. One-sided Jacobi (orange) gives errors around $\kappa\varepsilon$ — as good as the data allow, since rounding the entries of A already perturbs $\sigma_{\min}$ by about $\varepsilon\sigma_1$.

Try it yourself.

1. Add a counter to `svd_jacobi` and record the number of sweeps for random $n \times n$ matrices, $n = 10, 20, 40, 80$. (Typically 6–10.) Why is the total work nevertheless $O(n^3)$ per sweep?
2. Construct a matrix whose singular value $\sigma = 2$ is repeated. Compare the singular vectors from your SVD and from NumPy. Do they agree? Should they?

CHAPTER 5

The Pseudo-Inverse

Textbook: Chapter 9. **Assignment:** A5.

5.1 Implementation

The pseudo-inverse is one line once we have the SVD: invert the singular values above a tolerance, zero the rest, and form $V\Sigma^{+}U^{\mathsf{T}}$. Multiplying `Vt.T * s_inv` scales the columns of V by $1/\sigma_i$ through broadcasting — the same as $V\Sigma^{+}$ without building a diagonal matrix.

```
def pinv(A, rcond=None):
    """Moore-Penrose pseudo-inverse  A^+ = V Sigma^+ U^T.

    Singular values <= rcond * sigma_1 are treated as zero.
    Default rcond = max(m, n) * eps  (the same threshold as matrix rank).
    """
    A = _as_float_matrix(A)
    m, n = A.shape
    U, s, Vt = svd(A)
    if rcond is None:
        rcond = max(m, n) * EPS
    if s.size == 0 or s[0] == 0.0:
        return np.zeros((n, m))
    keep = s > rcond * s[0]
    s_inv = np.zeros_like(s)
    s_inv[keep] = 1.0 / s[keep]
    # V Sigma^+ U^T = sum_i (1/sigma_i) v_i u_i^T ; broadcasting scales the columns of V
    return (Vt.T * s_inv) @ U.T


def pinv_full_rank(A):
    """Pseudo-inverse from the full-rank formulas (no SVD needed).

    independent columns:  A^+ = (A^T A)^{-1} A^T      (left inverse)
    independent rows:     A^+ = A^T (A A^T)^{-1}      (right inverse)
    """
    A = _as_float_matrix(A)
    m, n = A.shape
    r = rank_by_elimination(A)
    if r == n:
        return solve(A.T @ A, A.T)
    if r == m:
        return solve(A @ A.T, A).T             # (A A^T)^{-1} is symmetric
    raise ValueError("A is rank deficient: use the SVD-based pinv")


def penrose_residuals(A, X):
```

```
    """Frobenius norms of the four Penrose-condition residuals (all ~0 for X = A^+)."""
    A = _as_float_matrix(A)
    X = _as_float_matrix(X)
    AX, XA = A @ X, X @ A
    fro = lambda M: float(np.sqrt(np.sum(M * M)))
    return (fro(A @ X @ A - A), fro(X @ A @ X - X), fro(AX - AX.T), fro(XA - XA.T))
```

`pinv_full_rank` implements the left- and right-inverse formulas of textbook Theorem 9.3 using our QR solver; it refuses rank-deficient input, because then $A^{\mathsf{T}}A$ and AA^{T} are singular. `penrose_residuals` measures how well a candidate satisfies the four Penrose conditions.

demos/d05_pinv.py

```
import numpy as np
from linalg_scratch import pinv, pinv_full_rank, penrose_residuals

np.set_printoptions(precision=4, suppress=True)
A = np.array([[3, 2, 2], [2, 3, -2]])                    # Worked Example 9.2
print("45 * A^+ =\n", 45 * pinv(A))
print("full-rank formula agrees:", np.allclose(pinv(A), pinv_full_rank(A)))

T = np.array([[1, 0], [0, 1], [1, 1]])                   # Worked Example 9.3
print("3 * T^+ =\n", 3 * pinv(T))
print("T^+ T =\n", pinv(T) @ T)

D = np.array([[1, 2], [2, 4], [3, 6]]); b = np.array([1, 2, 4])   # Worked Ex. 9.4
Dp = pinv(D)
print("70 * D^+ =\n", 70 * Dp)
x = Dp @ b
print("x+ = %s   (17/70 = %.4f, 17/35 = %.4f)" % (x, 17 / 70, 17 / 35))
print("residual =", b - D @ x)
print("Penrose residuals:", ["%.1e" % r for r in penrose_residuals(D, Dp)])
print("matches NumPy:", np.allclose(Dp, np.linalg.pinv(D)))
try:
    pinv_full_rank(D)
except ValueError as err:
    print("pinv_full_rank(D) ->", err)
```

OUTPUT

```
45 * A^+ =
 [[  7.   2.]
 [  2.   7.]
 [ 10. -10.]]
full-rank formula agrees: True
3 * T^+ =
 [[ 2. -1.  1.]
 [-1.  2.  1.]]
T^+ T =
 [[1. 0.]
 [0. 1.]]
70 * D^+ =
 [[1. 2. 3.]
 [2. 4. 6.]]
x+ = [0.2429 0.4857]   (17/70 = 0.2429, 17/35 = 0.4857)
```

```
residual = [-0.2143 -0.4286  0.3571]
Penrose residuals: ['0.0e+00', '0.0e+00', '0.0e+00', '0.0e+00']
matches NumPy: True
pinv_full_rank(D) -> A is rank deficient: use the SVD-based pinv
```

Each result matches the textbook: $45A^{+} = \begin{bmatrix} 7 & 2 \\ 2 & 7 \\ 10 & -10 \end{bmatrix}$ (Worked Example 9.2), $3T^{+} = \begin{bmatrix} 2 & -1 & 1 \\ -1 & 2 & 1 \end{bmatrix}$ with $T^{+}T = I$ (Worked Example 9.3), and for the rank-deficient system $70D^{+} = D^{\mathsf{T}}$, $\boldsymbol{x}^{+} = (17/70, 17/35)$ with residual $(-3/14, -3/7, 5/14)$ (Worked Example 9.4).

5.2 The tolerance matters

Textbook Section 9.6 showed that the pseudo-inverse is discontinuous. Here is what that looks like numerically for $A_\varepsilon = \mathrm{diag}(1, \varepsilon)$:

demos/d05_discontinuity.py

```python
import numpy as np
import matplotlib.pyplot as plt
from plotstyle import C1, C2, savefig
from linalg_scratch import pinv

eps_values = np.logspace(-16, 0, 33)
size_default, size_tol = [], []
for e in eps_values:
    A = np.array([[1.0, 0.0], [0.0, e]])
    size_default.append(np.abs(pinv(A)).max())          # rcond = 2 * machine eps
    size_tol.append(np.abs(pinv(A, rcond=1e-6)).max())   # rcond = 1e-6
plt.loglog(eps_values, size_default, color=C1, label="default rcond
    ↪ $=2\\varepsilon_{mach}$")
plt.loglog(eps_values, size_tol, color=C2, ls="--", label="rcond $=10^{-6}$")
plt.xlabel("$\\varepsilon$ in $A_\\varepsilon=$diag$(1,\\varepsilon)$")
plt.ylabel("largest entry of $A_\\varepsilon^+$"); plt.legend(); savefig("pinv_jump")
for e in (1e-3, 1e-7, 1e-17):
    A = np.array([[1.0, 0.0], [0.0, e]])
    print("eps=%.0e  default: %s   rcond=1e-6: %s"
          % (e, np.diag(pinv(A)), np.diag(pinv(A, rcond=1e-6))))
```

OUTPUT

```
eps=1e-03  default: [   1. 1000.]   rcond=1e-6: [   1. 1000.]
eps=1e-07  default: [1.e+00 1.e+07]   rcond=1e-6: [1. 0.]
eps=1e-17  default: [1. 0.]   rcond=1e-6: [1. 0.]
```

There is no universally right tolerance. The default $(\max(m, n)\,\varepsilon_{\text{mach}}$, relative to $\sigma_1)$ only removes what is certainly rounding noise. If your data have, say, 4 significant digits, singular values below $10^{-4}\sigma_1$ are noise too, and `rcond=1e-4` is the honest choice.

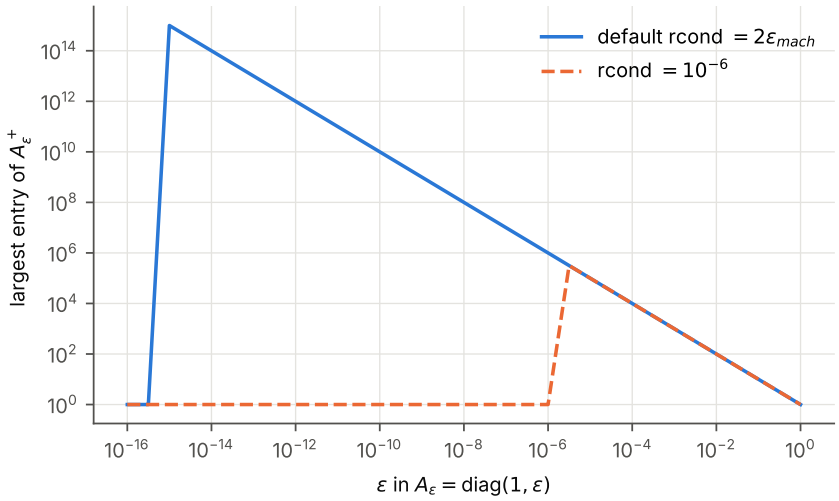


Figure 5.1: Largest entry of A_ε^+ as $\varepsilon \to 0$. It grows like $1/\varepsilon$ until ε falls below the tolerance, then jumps to 1. A larger `rcond` moves the jump to the right: it decides which singular values you trust.

Try it yourself.

1. Verify numerically that `pinv(pinv(A))` returns A and that `pinv(A.T)` equals `pinv(A).T` for random rectangular and rank-deficient matrices.
2. Find matrices A, B with $(AB)^+ \neq B^+A^+$ using your code (textbook Exercise 9.9 has one).

CHAPTER 6

Least Squares and Regularisation

Textbook: Chapters 8 and 10. **Assignments:** A5, A6.

6.1 Three solvers

For the normal equations we use the **Cholesky factorisation** $A^{\mathsf{T}}A = LL^{\mathsf{T}}$ (L lower triangular), which exists exactly when $A^{\mathsf{T}}A$ is positive definite, followed by two triangular solves.

```
def cholesky(S):
    """Cholesky factor L (lower triangular) with S = L L^T, for S positive definite."""
    S = _as_float_matrix(S)
    n = S.shape[0]
    L = np.zeros((n, n))
    for j in range(n):
        d = S[j, j] - L[j, :j] @ L[j, :j]
        if d <= 0.0:
            raise ValueError("matrix is not positive definite")
        L[j, j] = np.sqrt(d)
        L[j + 1:, j] = (S[j + 1:, j] - L[j + 1:, :j] @ L[j, :j]) / L[j, j]
    return L
```

```
def lstsq_normal(A, b):
    """Least squares via the normal equations A^T A x = A^T b (Cholesky solve)."""
    A = _as_float_matrix(A)
    L = cholesky(A.T @ A)
    y = forward_substitution(L, A.T @ np.asarray(b, dtype=float))
    return back_substitution(L.T, y)


def lstsq_qr(A, b):
    """Least squares via Householder QR:  R_1 x = (Q^T b)_1."""
    A = _as_float_matrix(A)
    n = A.shape[1]
    Q, R = householder_qr(A)
    return back_substitution(R[:n, :n], (Q.T @ np.asarray(b, dtype=float))[:n])


def lstsq_svd(A, b, rcond=None):
    """Minimum-norm least-squares solution  x = sum_i (u_i^T b / sigma_i) v_i = A^+ b."""
    return pinv(A, rcond) @ np.asarray(b, dtype=float)
```

demos/d06_linefit.py

```python
import numpy as np
from linalg_scratch import lstsq_normal, lstsq_qr, lstsq_svd

np.set_printoptions(precision=4, suppress=True)
A = np.array([[1, 0], [1, 1], [1, 2]]); b = np.array([6, 0, 0])   # Worked Ex. 8.1
for f in (lstsq_normal, lstsq_qr, lstsq_svd):
    print(f"{f.__name__:13s} C, D = {f(A, b)}")

A1 = np.array([[1, 2]]); print("min-norm solution of x1 + 2 x2 = 5:", lstsq_svd(A1, [5]))

# parabola fit, Exercise 8.4
t = np.array([-1, 0, 1, 2]); y = np.array([4, 6, 0, 6])
V = np.column_stack([np.ones(4), t, t ** 2])
c = lstsq_qr(V, y)
print("parabola coefficients:", c, "  residual:", y - V @ c)
```

OUTPUT

```
lstsq_normal  C, D = [ 5. -3.]
lstsq_qr      C, D = [ 5. -3.]
lstsq_svd     C, D = [ 5. -3.]
min-norm solution of x1 + 2 x2 = 5: [1. 2.]
parabola coefficients: [ 3. -1.  1.]   residual: [-1.  3. -3.  1.]
```

All three solvers find the line $y = 5 - 3t$ of textbook Worked Example 8.1, the minimum-norm solution $(1, 2)$ of Worked Example 8.4, and the parabola $y = 3 - t + t^2$ of Exercise 8.4 with residual $(-1, 3, -3, 1)$.

6.2 When the methods disagree

Fitting a polynomial of degree d with the monomial basis $1, t, \dots, t^d$ gives notoriously ill-conditioned Vandermonde matrices. Here we generate exact data from the polynomial with all coefficients 1 and ask each method to recover the coefficients.

demos/d06_polyfit.py

```python
import numpy as np
import matplotlib.pyplot as plt
from plotstyle import C1, C2, C3, savefig
from linalg_scratch import vandermonde, lstsq_normal, lstsq_qr, lstsq_svd

t = np.linspace(0, 1, 50)
degrees = range(1, 17)
errs = {"normal equations": [], "Householder QR": [], "SVD": []}
for d in degrees:
    A = vandermonde(t, d)
    c_true = np.ones(d + 1)                    # exact data from a known polynomial
    b = A @ c_true
    for (name, f) in zip(errs, (lstsq_normal, lstsq_qr, lstsq_svd)):
        try:
            c = f(A, b)
            errs[name].append(np.abs(c - c_true).max())
```

```
        except ValueError:                    # Cholesky breaks down
            errs[name].append(np.nan)
for (name, e), col, mk in zip(errs.items(), (C1, C2, C3), ("o", "s", "^")):
    plt.semilogy(list(degrees), e, marker=mk, color=col, label=name)
plt.xlabel("polynomial degree"); plt.ylabel("max error in coefficients")
plt.legend(); savefig("polyfit_methods")
print("degree  kappa(A)    normal eq.   QR        SVD")
for d in (4, 8, 12, 16):
    s = np.linalg.svd(vandermonde(t, d), compute_uv=False)
    print("%4d   %9.1e   %9.1e  %9.1e  %9.1e" % (d, s[0] / s[-1],
          errs["normal equations"][d - 1], errs["Householder QR"][d - 1], errs["SVD"][d
    ↪ - 1]))
```

OUTPUT

```
degree  kappa(A)    normal eq.   QR        SVD
   4     6.4e+02     3.3e-12    3.7e-14    8.5e-14
   8     6.3e+05     2.1e-06    4.7e-12    2.9e-11
  12     6.8e+08     7.0e+00    3.5e-08    4.2e-07
  16     8.2e+11         nan    8.4e-05    5.3e-05
```

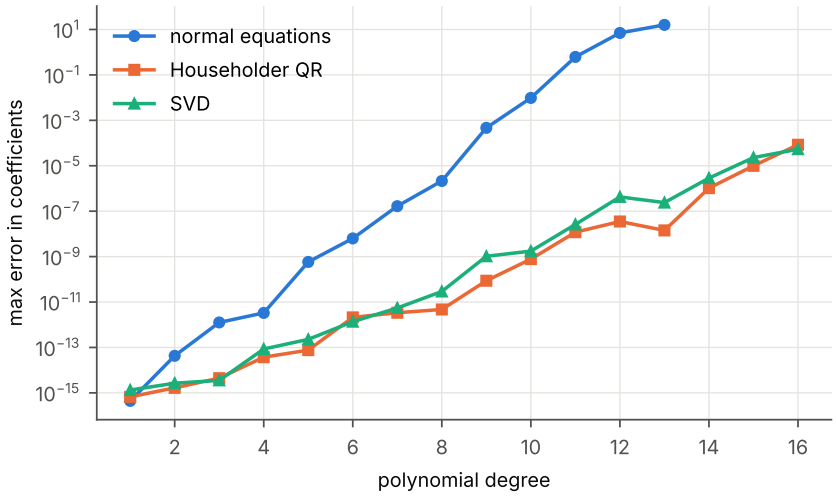


Figure 6.1: Error in the fitted coefficients. The normal equations lose accuracy twice as fast (error $\sim \kappa^2 \varepsilon$) and Cholesky fails outright beyond degree 13, when $A^{\mathsf{T}}A$ is no longer numerically positive definite. QR and SVD track each other ($\sim \kappa\varepsilon$).

This is textbook Exercise 7.5 made concrete: $\kappa(A^{\mathsf{T}}A) = \kappa(A)^2$. At degree 12, $\kappa(A) \approx 7 \times 10^8$; the normal equations have *no* correct digits while QR still has about 7.

6.3 Regularising an ill-posed problem

```
def tsvd_solve(A, b, k, usv=None):
    """Truncated-SVD solution: keep only the k largest singular values.

    Pass usv=(U, s, Vt) to reuse an SVD you already computed (e.g. when
    trying many values of k).
    """
    U, s, Vt = svd(A) if usv is None else usv
    c = U[:, :k].T @ np.asarray(b, dtype=float)       # coefficients u_i^T b
    return Vt[:k].T @ (c / s[:k])


def ridge_solve(A, b, lam, usv=None):
    """Tikhonov / ridge solution  x = sum_i sigma_i/(sigma_i^2 + lam) (u_i^T b) v_i."""
    U, s, Vt = svd(A) if usv is None else usv
    c = U.T @ np.asarray(b, dtype=float)
    return Vt.T @ (s / (s ** 2 + lam) * c)
```

Our test problem is *deblurring*. A signal $\boldsymbol{x}$ on 100 points is blurred by a Gaussian (each output is a weighted average of nearby inputs) and a little noise of size 10^{-3} is added. Recovering $\boldsymbol{x}$ from the blurred data $\boldsymbol{b}$ means solving $A\boldsymbol{x} = \boldsymbol{b}$ with a very ill-conditioned A.

demos/d06_regularisation.py

```
import numpy as np
import matplotlib.pyplot as plt
from plotstyle import C1, C2, C3, INK2, savefig
from linalg_scratch import svd, tsvd_solve, ridge_solve

# a 1-D blurring operator: each output is a Gaussian-weighted average
n = 100
s_grid = np.linspace(0, 1, n)
A = np.exp(-((s_grid[:, None] - s_grid[None, :]) ** 2) / (2 * 0.02 ** 2))
A /= A.sum(axis=1, keepdims=True)
x_true = ((s_grid > 0.2) & (s_grid < 0.45)) * 1.0 + np.sin(np.pi * s_grid) * (s_grid >
    ↪ 0.55)
rng = np.random.default_rng(0)
b = A @ x_true + 1e-3 * rng.standard_normal(n)          # blurred + noise

usv = U, s, Vt = svd(A)                                 # compute the SVD once
print("sigma_1 = %.2e, sigma_n = %.2e, kappa = %.1e" % (s[0], s[-1], s[0] / s[-1]))
err = lambda x: np.linalg.norm(x - x_true) / np.linalg.norm(x_true)
naive = Vt.T @ ((U.T @ b) / s)                          # A^{-1} b = sum (u_i^T b /
    ↪ sigma_i) v_i
print("naive A^+ b      relative error: %.1e" % err(naive))
best_k = min(range(1, n + 1), key=lambda k: err(tsvd_solve(A, b, k, usv)))
lams = np.logspace(-8, 0, 41)
best_lam = min(lams, key=lambda l: err(ridge_solve(A, b, l, usv)))
print("TSVD,  best k   = %3d    relative error: %.3f" % (best_k, err(tsvd_solve(A, b,
    ↪ best_k, usv))))
print("ridge, best lam = %.0e  relative error: %.3f" % (best_lam, err(ridge_solve(A, b,
    ↪ best_lam, usv))))

```

```
fig, ax = plt.subplots(1, 2, figsize=(7.2, 3.0))
ax[0].plot(s_grid, x_true, color=INK2, lw=2.4, label="true signal")
ax[0].plot(s_grid, b, color="#9a9890", lw=1, label="blurred + noise")
ax[0].plot(s_grid, tsvd_solve(A, b, best_k, usv), color=C1, label=f"TSVD, k={best_k}")
ax[0].plot(s_grid, ridge_solve(A, b, best_lam, usv), color=C2, ls="--", label="ridge")
ax[0].legend(fontsize=7); ax[0].set_xlabel("position")
ax[1].semilogy(s, "o", ms=2.5, color=C1, label="$\\sigma_i$")
ax[1].semilogy(np.abs(U.T @ b), "s", ms=2.5, color=C2, label="$|u_i^Tb|$")
ax[1].set_xlabel("index $i$"); ax[1].legend(fontsize=7)
savefig("regularisation")
```

OUTPUT

```
sigma_1 = 1.00e+00, sigma_n = 8.65e-09, kappa = 1.2e+08
naive A^+ b      relative error: 2.8e+04
TSVD,  best k   =  53    relative error: 0.100
ridge, best lam = 4e-05  relative error: 0.102
```

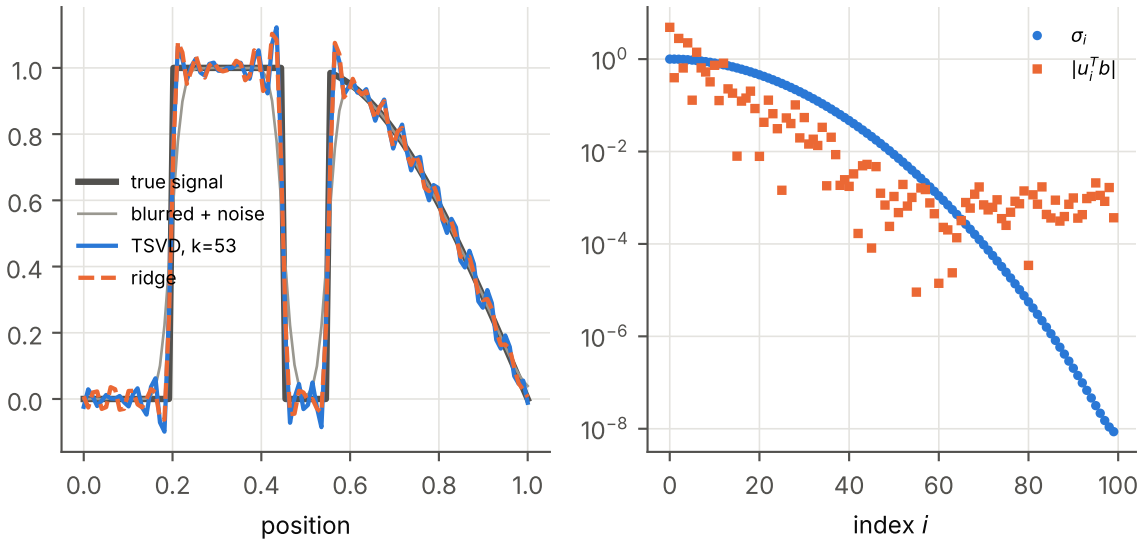


Figure 6.2: Left: true signal, blurred noisy data, and the TSVD and ridge reconstructions (nearly indistinguishable). Right: the singular values σ_i fall to 10^{-8}, while the data coefficients $\left|\boldsymbol{u}_i^{\mathsf{T}}\boldsymbol{b}\right|$ stop falling at the noise level $\approx 10^{-3}$. Dividing noise by tiny σ_i is what destroys the naive solution.

The right panel is called a **Picard plot**, and it explains everything. For $i \lesssim 50$ the coefficients $\boldsymbol{u}_i^{\mathsf{T}}\boldsymbol{b}$ decay along with σ_i: they carry information about the signal. Beyond that they flatten out at the noise level, while σ_i keeps falling — so the terms $\boldsymbol{u}_i^{\mathsf{T}}\boldsymbol{b}/\sigma_i$ explode. The naive solution has relative error 3×10^4; truncating at $k = 53$ (or ridge with $\lambda = 4 \times 10^{-5}$) gives about 10%. Both regularised solutions blur the sharp edges slightly: that is the price of discarding the high-frequency directions where signal and noise cannot be told apart.

Try it yourself.

1. Raise the noise to 10^{-2}. How do the best k and the best λ change? Explain using the Picard plot.
2. In practice $\boldsymbol{x}_{\text{true}}$ is unknown, so you cannot choose k by comparing with it. Implement the *discrepancy principle*: choose the smallest k with $\|A\boldsymbol{x}_k - \boldsymbol{b}\| \leq \|\text{noise}\|$. How close does it get to the best k?

CHAPTER 7

Applications: Norms, Low Rank, PCA

Textbook: Chapters 7 and 10. **Assignment:** A6.

7.1 Norms, condition number, rank

```python
def singular_values(A):
    return svd(A)[1]


def spectral_norm(A):
    s = singular_values(A)
    return float(s[0]) if s.size else 0.0


def frobenius_norm(A):
    A = _as_float_matrix(A)
    return float(np.sqrt(np.sum(A * A)))


def nuclear_norm(A):
    return float(np.sum(singular_values(A)))


def condition_number(A):
    """kappa = sigma_1 / sigma_r (smallest NON-ZERO singular value); inf if A = 0."""
    s = singular_values(A)
    s = s[s > max(np.shape(A)) * EPS * (s[0] if s.size else 0.0)]
    return float(s[0] / s[-1]) if s.size else np.inf


def matrix_rank(A, tol=None):
    s = singular_values(A)
    if s.size == 0:
        return 0
    if tol is None:
        tol = max(np.shape(A)) * EPS * s[0]
    return int(np.sum(s > tol))
```

demos/d07_norms.py

```python
import numpy as np
from linalg_scratch import (spectral_norm, frobenius_norm, nuclear_norm,
                            condition_number, matrix_rank, low_rank_approx, svd)

A = np.array([[3, 0], [4, 5]])                          # Worked Example 7.2
print("||A||_2 = %.4f  ||A||_F = %.4f  ||A||_* = %.4f  kappa = %.4f  rank = %d"
      % (spectral_norm(A), frobenius_norm(A), nuclear_norm(A),
         condition_number(A), matrix_rank(A)))

B = np.array([[1, 1], [1, 1.0001]])                     # Worked Example 7.4
print("kappa(B) = %.1f" % condition_number(B))

# Eckart-Young: the truncated SVD beats every other rank-k matrix
rng = np.random.default_rng(7)
M = rng.standard_normal((30, 20))
U, s, Vt = svd(M)
for k in (1, 5, 10):
    Mk = low_rank_approx(M, k)
    other = rng.standard_normal((30, k)) @ rng.standard_normal((k, 20))
    other *= np.sum(M * other) / np.sum(other * other)    # best scaling of a random
    ↪ rank-k matrix
    print("k=%2d  ||M-M_k||_2 = %.4f  (sigma_%d = %.4f)   random rank-k error: %.4f"
          % (k, spectral_norm(M - Mk), k + 1, s[k], spectral_norm(M - other)))
```

OUTPUT

```
||A||_2 = 6.7082  ||A||_F = 7.0711  ||A||_* = 8.9443  kappa = 3.0000  rank = 2
kappa(B) = 40002.0
k= 1  ||M-M_k||_2 = 8.0470  (sigma_2 = 8.0470)   random rank-k error: 8.6650
k= 5  ||M-M_k||_2 = 6.5106  (sigma_6 = 6.5106)   random rank-k error: 8.6505
k=10  ||M-M_k||_2 = 4.1738  (sigma_11 = 4.1738)   random rank-k error: 8.6623
```

The first two lines reproduce textbook Worked Examples 7.2 and 7.4 ($\|A\|_2 = 3\sqrt{5}$, $\|A\|_F = \sqrt{50}$, $\|A\|_* = 4\sqrt{5}$, $\kappa = 3$; $\kappa \approx 40{,}002$). The rest checks the Eckart–Young theorem: for each k the truncation error equals σ_{k+1} exactly, and a random rank-k matrix — even optimally scaled — does much worse.

7.2 Low-rank approximation of an image

```python
def low_rank_approx(A, k):
    """Best rank-k approximation (Eckart-Young): sum_{i<=k} sigma_i u_i v_i^T."""
    U, s, Vt = svd(A)
    k = min(k, s.size)
    return (U[:, :k] * s[:k]) @ Vt[:k]


def pca(X, k):
    """Principal component analysis of the rows of X (N samples x d features).

    Returns (components (k x d), explained_variance (k,), scores (N x k), mean).
    """
```

```
    X = _as_float_matrix(X)
    mean = X.mean(axis=0)
    Xc = X - mean                                   # centre the data
    U, s, Vt = svd(Xc)
    N = X.shape[0]
    explained_variance = s[:k] ** 2 / (N - 1)
    scores = U[:, :k] * s[:k]                       # = Xc @ Vt[:k].T
    return Vt[:k], explained_variance, scores, mean
```

A greyscale image is just a matrix. We generate a synthetic 180×270 "photograph" (sky gradient, sun, hills, buildings, a tiled road and a little sensor noise) and approximate it with the truncated SVD.

demos/d07_image.py

```
import numpy as np
import matplotlib.pyplot as plt
from plotstyle import C1, savefig
from linalg_scratch import svd

# a synthetic 'photograph': sky gradient, sun, hills, buildings, tiled road
h, w = 180, 270
y, x = np.mgrid[0:h, 0:w] / np.array([h, w])[:, None, None]
img = 0.85 - 0.45 * y
img[(x - 0.75) ** 2 + (y - 0.22) ** 2 < 0.006] = 1.0
img[y > 0.55 + 0.08 * np.sin(9 * x)] = 0.45
for x0, x1, top in [(0.08, 0.2, 0.35), (0.24, 0.31, 0.25), (0.42, 0.55, 0.4)]:
    img[(x > x0) & (x < x1) & (y > top) & (y < 0.75)] = 0.2
img[(y > 0.4 - 0.8 * np.abs(x - 0.85)) & (y < 0.75) & (np.abs(x - 0.85) < 0.1)] = 0.35
    ↪ # a roof
tiles = 0.3 + 0.25 * ((np.floor(x * 24) + np.floor(y * 24)) % 2)
road = y > 0.82
img[road] = tiles[road]
img += 0.01 * np.random.default_rng(0).standard_normal(img.shape)   # sensor noise
img = np.clip(img, 0, 1)

U, s, Vt = svd(img)
ks = (2, 10, 30)
fig, ax = plt.subplots(1, 4, figsize=(8.0, 1.75))
for a, k in zip(ax, (None,) + ks):
    shown = img if k is None else (U[:, :k] * s[:k]) @ Vt[:k]
    a.imshow(shown, cmap="gray", vmin=0, vmax=1); a.axis("off")
    a.set_title("original" if k is None else f"rank {k}", fontsize=8)
savefig("image_compression")
plt.figure(figsize=(6.0, 2.6))
plt.semilogy(range(1, s.size + 1), s, color=C1)
plt.xlabel("$i$"); plt.ylabel("$\\sigma_i$"); savefig("image_singular_values")

total = np.sum(s ** 2)
print("image size %d x %d = %d numbers" % (h, w, h * w))
print("  k   storage   % of original   rel. Frobenius error")
for k in (1, 2, 5, 10, 20, 30, 50):
    store = k * (h + w + 1)
    print("%3d  %8d   %6.1f%%          %.4f"
```

```
39          % (k, store, 100 * store / (h * w), np.sqrt(np.sum(s[k:] ** 2) / total)))
```

OUTPUT

```
image size 180 x 270 = 48600 numbers
  k   storage   % of original   rel. Frobenius error
  1       451      0.9%          0.1939
  2       902      1.9%          0.1334
  5      2255      4.6%          0.0569
 10      4510      9.3%          0.0300
 20      9020     18.6%          0.0211
 30     13530     27.8%          0.0175
 50     22550     46.4%          0.0132
```

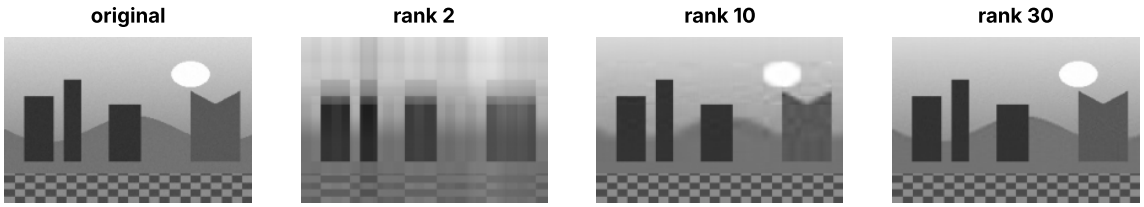


Figure 7.1: The image and its rank-2, rank-10 and rank-30 approximations. Horizontal and vertical structure (buildings, the sky gradient) is captured by very few terms; curved and diagonal edges (sun, hills, roof) need more.

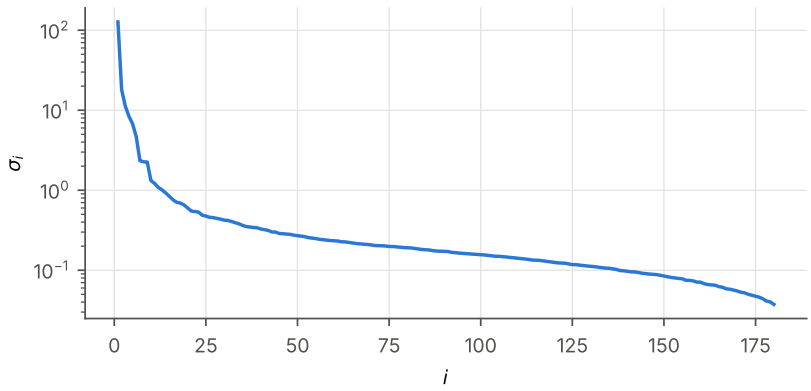


Figure 7.2: Singular values of the image: a few large ones carry the scene; the long flat tail is mostly the added noise.

With $k = 10$ the approximation uses 9.3% of the storage with a relative error of 3%. Beyond $k \approx 20$ the error decreases slowly, because the remaining singular values describe noise — which the truncation is actually removing (a simple form of denoising).

7.3 Principal component analysis

demos/d07_pca.py

```python
import numpy as np
import matplotlib.pyplot as plt
from plotstyle import C1, C2, INK2, savefig
from linalg_scratch import pca

np.set_printoptions(precision=4, suppress=True)
X = np.array([[-2, -1], [-1, -2], [1, 2], [2, 1], [0, 0]])   # Worked Example 10.4
comps, var, scores, mean = pca(X, 2)
print("principal directions (rows):\n", comps)
print("variances:", var, "  explained:", var / var.sum())
print("scores on PC1:", scores[:, 0])

rng = np.random.default_rng(0)                          # a larger, noisier cloud
Z = rng.standard_normal((300, 2)) * [2.0, 0.5]
th = np.radians(30)
D = Z @ np.array([[np.cos(th), np.sin(th)], [-np.sin(th), np.cos(th)]]) + [1, 2]
comps, var, _, mean = pca(D, 2)
print("cloud: directions\n", comps, "\n std devs:", np.sqrt(var))
plt.figure(figsize=(4.2, 3.4))
plt.scatter(D[:, 0], D[:, 1], s=6, color="#9a9890")
for c, sd, col, lab in zip(comps, np.sqrt(var), (C1, C2), ("PC1", "PC2")):
    plt.annotate("", mean + 2 * sd * c, mean,
                 arrowprops=dict(arrowstyle="->", color=col, lw=2))
    plt.text(*(mean + 2.2 * sd * c), lab, color=INK2)
plt.axis("equal"); savefig("pca_cloud")
```

OUTPUT

```
principal directions (rows):
 [[ 0.7071  0.7071]
 [ 0.7071 -0.7071]]
variances: [4.5 0.5]   explained: [0.9 0.1]
scores on PC1: [-2.1213 -2.1213  2.1213  2.1213  0.    ]
cloud: directions
 [[ 0.8639  0.5037]
 [-0.5037  0.8639]]
 std devs: [2.0092 0.4941]
```

The first block reproduces textbook Worked Example 10.4: principal direction $\frac{1}{\sqrt{2}}(1,1)$, variances 4.5 and 0.5, 90% explained, scores ± 2.1213. The second block analyses 300 random points generated with standard deviations 2 and 0.5 along axes rotated by 30°; PCA recovers the direction $(\cos 30^\circ, \sin 30^\circ) \approx (0.866, 0.5)$ and the two standard deviations to within sampling error.

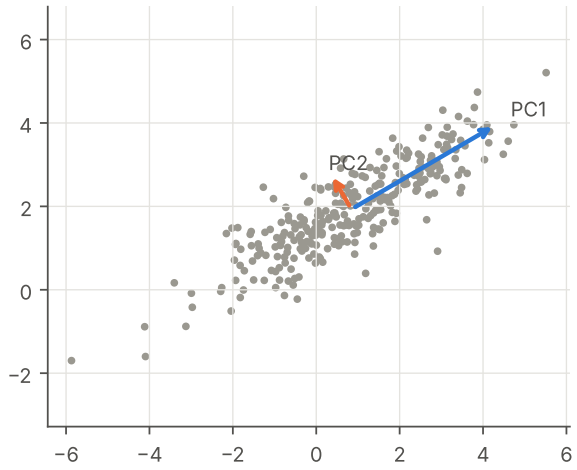


Figure 7.3: The point cloud with its principal axes, each drawn with length two standard deviations.

Try it yourself.

1. Load a real photograph (convert it to greyscale with `matplotlib.image.imread` and averaging the colour channels). Plot its singular values on a log scale and find the k that gives 1% relative error.
2. Write `pca_reconstruct(X, k)` that projects the data onto the first k principal components and maps them back. Show that the mean squared reconstruction error equals the sum of the discarded variances (times $(N-1)/N$).

Where Next: the Assignments

You have now seen every algorithm of the course implemented and tested. The Coding Assignments ask you to write them yourself, inside a fixed interface, with an automatic test suite checking correctness, edge cases (zero matrices, rank deficiency, wide and tall shapes, repeated singular values) and the "no `numpy.linalg`" rule. Several functions there go beyond this manual — the four fundamental subspaces from the SVD, the Eckart–Young error formula, compression ratios, explained variance — so the assignments are not just a re-typing exercise. Try each assignment before re-reading the matching part of this manual.

APPENDIX A

Complete Source of `linalg_scratch.py`

For reference, here is the whole library in one piece. The comment lines of the form `# [begin:...]` and `# [end:...]` mark the excerpts printed in the chapters.

```
"""
linalg_scratch.py -- the SVD and the pseudo-inverse, implemented from scratch.

Companion code for the "Python Lab Manual".  Every function uses only basic
NumPy array arithmetic (+, -, *, /, sqrt, sum, slicing, and the @ operator
once we have written our own matmul).  Nothing from numpy.linalg is used;
numpy.linalg appears only in the demo scripts, to *check* our answers.

Sections (match the parts of the Lab Manual):
    Part 1  vectors, matrices, Gaussian elimination
    Part 2  Gram-Schmidt, Householder QR, triangular solves, projections
    Part 3  eigenvalues: Rayleigh quotient, power iteration, Jacobi method
    Part 4  the SVD: via eig(A^T A) and by one-sided Jacobi
    Part 5  the pseudo-inverse and the Penrose conditions
    Part 6  least squares and regularisation
    Part 7  applications: norms, low-rank approximation, PCA
"""
import numpy as np

EPS = np.finfo(float).eps          # machine epsilon, about 2.22e-16


def _as_float_matrix(A):
    """Return A as a 2-D float array (a copy, so callers' data is never changed)."""
    A = np.array(A, dtype=float)
    if A.ndim == 1:
        A = A.reshape(-1, 1)
    return A


# ============================================================================
# Part 1 -- vectors and matrices
# ============================================================================

# [begin:dot]
def dot(u, v):
    """Dot product u . v = sum_i u_i v_i, written as an explicit loop."""
    u = np.asarray(u, dtype=float).ravel()
    v = np.asarray(v, dtype=float).ravel()
    if u.shape != v.shape:
        raise ValueError(f"shape mismatch: {u.shape} vs {v.shape}")
    total = 0.0
    for ui, vi in zip(u, v):
        total += ui * vi
    return total
# [end:dot]

```

```python

# [begin:norm]
def norm(x, p=2):
    """Vector norm: p = 1, 2 or np.inf.

    The 2-norm is computed as  s * sqrt(sum((x/s)^2))  with s = max|x_i|.
    Mathematically this is just sqrt(sum x_i^2); numerically, the scaling
    stops x_i^2 from overflowing when |x_i| is huge (e.g. 1e200).
    """
    x = np.asarray(x, dtype=float).ravel()
    if x.size == 0:
        return 0.0
    if p == 1:
        return float(np.sum(np.abs(x)))
    if p == np.inf:
        return float(np.max(np.abs(x)))
    if p == 2:
        s = np.max(np.abs(x))
        if s == 0.0:
            return 0.0
        return float(s * np.sqrt(np.sum((x / s) ** 2)))
    raise ValueError("p must be 1, 2 or np.inf")
# [end:norm]


# [begin:normalize]
def normalize(x):
    """Unit vector x / ||x||.  Raises ValueError for the zero vector."""
    x = np.asarray(x, dtype=float)
    n = norm(x)
    if n == 0.0:
        raise ValueError("cannot normalise the zero vector")
    return x / n


def angle(u, v):
    """Angle between u and v in radians, in [0, pi]."""
    c = dot(u, v) / (norm(u) * norm(v))
    return float(np.arccos(np.clip(c, -1.0, 1.0)))   # clip guards rounding


def project_onto_vector(b, a):
    """Projection of b onto the line through a:  (a.b / a.a) a."""
    a = np.asarray(a, dtype=float)
    return (dot(a, b) / dot(a, a)) * a
# [end:normalize]


# [begin:matmul]
def matmul(A, B):
    """Matrix product by the definition (AB)_ij = sum_k A_ik B_kj.

    Three nested loops: O(m n p) operations.  We write it once to see what
    the @ operator does; afterwards we use @, which runs the same arithmetic
    in optimised compiled code.
    """
    A = _as_float_matrix(A)
    B = _as_float_matrix(B)
    m, n = A.shape
```

```
    n2, p = B.shape
    if n != n2:
        raise ValueError(f"inner dimensions differ: {A.shape} @ {B.shape}")
    C = np.zeros((m, p))
    for i in range(m):
        for j in range(p):
            s = 0.0
            for k in range(n):
                s += A[i, k] * B[k, j]
            C[i, j] = s
    return C


def outer(u, v):
    """Outer product u v^T: the (len(u) x len(v)) matrix with entries u_i v_j."""
    u = np.asarray(u, dtype=float).ravel()
    v = np.asarray(v, dtype=float).ravel()
    return u[:, None] * v[None, :]
# [end:matmul]


# [begin:rref]
def rref(A, tol=1e-10):
    """Reduced row echelon form by Gauss-Jordan elimination.

    Partial pivoting: in each column we pick the row with the largest
    |entry| as pivot, which keeps the multipliers <= 1 in size.
    Entries smaller than tol * max|A| count as zero.

    Returns (R, pivot_columns).
    """
    R = _as_float_matrix(A)
    m, n = R.shape
    scale = np.max(np.abs(R)) if R.size else 0.0
    thresh = tol * max(scale, 1.0)
    pivots = []
    row = 0
    for col in range(n):
        if row >= m:
            break
        # choose the pivot row: largest entry in this column, at or below `row`
        p = row + int(np.argmax(np.abs(R[row:, col])))
        if abs(R[p, col]) <= thresh:
            R[row:, col] = 0.0          # whole column is (numerically) zero
            continue
        R[[row, p]] = R[[p, row]]        # swap rows
        R[row] = R[row] / R[row, col]    # scale pivot to 1
        for i in range(m):               # eliminate above and below
            if i != row:
                R[i] = R[i] - R[i, col] * R[row]
        pivots.append(col)
        row += 1
    R[np.abs(R) <= thresh] = 0.0
    return R, pivots


def rank_by_elimination(A, tol=1e-10):
    """Rank = number of pivots in the reduced row echelon form."""
    return len(rref(A, tol)[1])
```

```python


def null_space_by_elimination(A, tol=1e-10):
    """Basis of N(A): one 'special solution' per free variable (columns of the result)."""
    R, pivots = rref(A, tol)
    n = R.shape[1]
    free = [j for j in range(n) if j not in pivots]
    basis = []
    for f in free:
        x = np.zeros(n)
        x[f] = 1.0
        for i, pc in enumerate(pivots):  # pivot variable = -(its row's entry in column f)
            x[pc] = -R[i, f]
        basis.append(x)
    return np.array(basis).T if basis else np.zeros((n, 0))
# [end:rref]


# ======================================================================
# Part 2 -- orthogonalisation and QR
# ======================================================================

# [begin:gs]
def gram_schmidt_classical(A):
    """Classical Gram-Schmidt: A = Q R with Q (m x n) orthonormal columns.

    r_ij = q_i . a_j uses the ORIGINAL column a_j.
    """
    A = _as_float_matrix(A)
    m, n = A.shape
    Q = np.zeros((m, n))
    R = np.zeros((n, n))
    for j in range(n):
        w = A[:, j].copy()
        for i in range(j):
            R[i, j] = Q[:, i] @ A[:, j]       # project the original a_j
            w -= R[i, j] * Q[:, i]
        R[j, j] = norm(w)
        if R[j, j] == 0.0:
            raise ValueError("columns are linearly dependent")
        Q[:, j] = w / R[j, j]
    return Q, R


def gram_schmidt_modified(A):
    """Modified Gram-Schmidt: same result in exact arithmetic.

    r_ij = q_i . w uses the PARTIALLY REDUCED vector w, so each subtraction
    also removes the rounding errors made by the previous ones.
    """
    A = _as_float_matrix(A)
    m, n = A.shape
    Q = np.zeros((m, n))
    R = np.zeros((n, n))
    for j in range(n):
        w = A[:, j].copy()
        for i in range(j):
            R[i, j] = Q[:, i] @ w             # project the current w
            w -= R[i, j] * Q[:, i]
```

```
        R[j, j] = norm(w)
        if R[j, j] == 0.0:
            raise ValueError("columns are linearly dependent")
        Q[:, j] = w / R[j, j]
    return Q, R
# [end:gs]


# [begin:householder]
def householder_qr(A):
    """Full QR factorisation A = Q R by Householder reflections.

    Q is m x m orthogonal, R is m x n upper triangular.
    Step k reflects column k (from row k down) onto a multiple of e_1 with
    H = I - 2 v v^T / (v^T v),  v = x + sign(x_1) ||x|| e_1.
    The sign choice avoids subtracting two nearly equal numbers.
    """
    R = _as_float_matrix(A)
    m, n = R.shape
    Q = np.eye(m)
    for k in range(min(m - 1, n)):
        x = R[k:, k].copy()
        alpha = norm(x)
        if alpha == 0.0:
            continue                        # column already zero below the diagonal
        v = x.copy()
        v[0] += (1.0 if x[0] >= 0 else -1.0) * alpha
        vtv = v @ v
        # apply H = I - 2 v v^T / v^T v to the trailing rows of R ...
        R[k:, :] -= np.outer(v, (2.0 / vtv) * (v @ R[k:, :]))
        # ... and accumulate Q = H_1 H_2 ... H_k (apply H on the right)
        Q[:, k:] -= np.outer((Q[:, k:] @ v) * (2.0 / vtv), v)
        R[k + 1:, k] = 0.0                  # exact zeros below the diagonal
    return Q, R
# [end:householder]


# [begin:triangular]
def back_substitution(U, b):
    """Solve U x = b for upper-triangular U (b may be a vector or a matrix)."""
    U = _as_float_matrix(U)
    b = np.array(b, dtype=float)
    n = U.shape[0]
    x = np.zeros_like(b, dtype=float)
    for i in range(n - 1, -1, -1):
        if U[i, i] == 0.0:
            raise ValueError("singular triangular matrix")
        x[i] = (b[i] - U[i, i + 1:] @ x[i + 1:]) / U[i, i]
    return x


def forward_substitution(L, b):
    """Solve L x = b for lower-triangular L."""
    L = _as_float_matrix(L)
    b = np.array(b, dtype=float)
    n = L.shape[0]
    x = np.zeros_like(b, dtype=float)
    for i in range(n):
        if L[i, i] == 0.0:
```

```
            raise ValueError("singular triangular matrix")
        x[i] = (b[i] - L[i, :i] @ x[:i]) / L[i, i]
    return x


def solve(A, b):
    """Solve a square system A x = b via Householder QR: R x = Q^T b."""
    A = _as_float_matrix(A)
    if A.shape[0] != A.shape[1]:
        raise ValueError("solve() needs a square matrix")
    Q, R = householder_qr(A)
    return back_substitution(R, Q.T @ np.asarray(b, dtype=float))
# [end:triangular]


# [begin:projection]
def projection_matrix(A):
    """Orthogonal projector onto C(A) for A with independent columns: P = Q Q^T."""
    A = _as_float_matrix(A)
    n = A.shape[1]
    Q, _ = householder_qr(A)
    Qn = Q[:, :n]                # first n columns: orthonormal basis of C(A)
    return Qn @ Qn.T


def complete_orthonormal_basis(Q, tol=1e-10):
    """Extend the orthonormal columns of Q (m x k) to an m x m orthogonal matrix.

    Try the standard basis vectors e_1, e_2, ... in turn; remove their
    components along the columns we already have (twice, for safety --
    'twice is enough'); keep the result if anything substantial is left.
    """
    Q = _as_float_matrix(Q)
    m, k = Q.shape
    cols = [Q[:, j] for j in range(k)]
    for i in range(m):
        if len(cols) == m:
            break
        w = np.zeros(m)
        w[i] = 1.0
        for _ in range(2):
            for q in cols:
                w -= (q @ w) * q
        nw = norm(w)
        if nw > tol:
            cols.append(w / nw)
    return np.column_stack(cols) if cols else np.zeros((m, 0))
# [end:projection]


# ======================================================================
# Part 3 -- eigenvalues of symmetric matrices
# ======================================================================

# [begin:power]
def rayleigh_quotient(S, x):
    """R(x) = x^T S x / x^T x."""
    x = np.asarray(x, dtype=float)
    return float(x @ (S @ x) / (x @ x))
```

```python


def power_iteration(S, x0=None, tol=1e-12, max_iter=10_000, seed=0):
    """Dominant eigenpair of a symmetric matrix S.

    Returns (lam, v, history) where history[k] is the Rayleigh quotient
    after k+1 steps.  Converges when |lambda_1| > |lambda_2|.
    """
    S = _as_float_matrix(S)
    n = S.shape[0]
    if x0 is None:
        x0 = np.random.default_rng(seed).standard_normal(n)
    x = normalize(x0)
    lam = rayleigh_quotient(S, x)
    history = []
    for _ in range(max_iter):
        y = S @ x
        ny = norm(y)
        if ny == 0.0:                # x is in the null space: eigenvalue 0
            return 0.0, x, history + [0.0]
        x = y / ny
        lam_new = rayleigh_quotient(S, x)
        history.append(lam_new)
        if abs(lam_new - lam) <= tol * max(1.0, abs(lam_new)):
            lam = lam_new
            break
        lam = lam_new
    return lam, x, history


def eigs_by_deflation(S, k, **kwargs):
    """Top-k eigenpairs of symmetric PSD S: power iteration + deflation.

    After finding (lam, v) we replace S by S - lam v v^T, which moves that
    eigenvalue to 0 and leaves the others unchanged.
    """
    S = _as_float_matrix(S).copy()
    lams, vecs = [], []
    for _ in range(k):
        lam, v, _ = power_iteration(S, **kwargs)
        lams.append(lam)
        vecs.append(v)
        S = S - lam * np.outer(v, v)
    return np.array(lams), np.column_stack(vecs)
# [end:power]


# [begin:jacobi]
def jacobi_eigh(S, tol=1e-14, max_sweeps=60, return_history=False):
    """All eigenvalues and eigenvectors of a symmetric matrix (cyclic Jacobi).

    Each step picks an off-diagonal entry s_pq and applies a plane rotation
    J (in the p,q plane) with  S <- J^T S J  chosen so that the new s_pq = 0.
    One 'sweep' visits every pair p < q once.  The off-diagonal mass
    off(S) = sqrt(sum_{i != j} s_ij^2) decreases to zero (quadratically
    near the end), and the product of all rotations holds the eigenvectors.

    Returns (eigenvalues sorted in DECREASING order, V with eigenvectors as
    columns) -- and the list of off(S) after each sweep if requested.
```

```
    """
    A = _as_float_matrix(S).copy()
    n = A.shape[0]
    if A.shape != (n, n):
        raise ValueError("matrix must be square")
    if not np.allclose(A, A.T, atol=1e-12 * max(1.0, np.max(np.abs(A)))):
        raise ValueError("matrix must be symmetric")
    V = np.eye(n)
    history = []

    def off(M):
        # sum the off-diagonal squares directly; computing sum(M*M) - sum(diag^2)
        # would cancel catastrophically once the off-diagonal part is tiny
        O = M - np.diag(np.diag(M))
        return np.sqrt(np.sum(O * O))

    scale = np.sqrt(np.sum(A * A)) or 1.0
    for _ in range(max_sweeps):
        history.append(off(A))
        if off(A) <= tol * scale:
            break
        for p in range(n - 1):
            for q in range(p + 1, n):
                apq = A[p, q]
                if abs(apq) <= 1e-300:
                    continue
                # rotation angle: t = tan(theta) solves t^2 + 2 t theta_ - 1 = 0
                theta = (A[q, q] - A[p, p]) / (2.0 * apq)
                if abs(theta) > 1e150:             # avoid overflow in theta^2
                    t = 0.5 / theta
                else:
                    t = (1.0 if theta >= 0 else -1.0) / (abs(theta) + np.sqrt(theta * theta + 1.0))
                c = 1.0 / np.sqrt(t * t + 1.0)
                s = t * c
                # A <- A J  (columns p and q)
                Ap, Aq = A[:, p].copy(), A[:, q].copy()
                A[:, p] = c * Ap - s * Aq
                A[:, q] = s * Ap + c * Aq
                # A <- J^T A  (rows p and q)
                Ap, Aq = A[p, :].copy(), A[q, :].copy()
                A[p, :] = c * Ap - s * Aq
                A[q, :] = s * Ap + c * Aq
                A[p, q] = A[q, p] = 0.0        # exactly zero by construction
                # accumulate eigenvectors V <- V J
                Vp, Vq = V[:, p].copy(), V[:, q].copy()
                V[:, p] = c * Vp - s * Vq
                V[:, q] = s * Vp + c * Vq
    else:
        history.append(off(A))
    eigvals = np.diag(A).copy()
    order = np.argsort(eigvals)[::-1]          # largest first
    eigvals, V = eigvals[order], V[:, order]
    return (eigvals, V, history) if return_history else (eigvals, V)
# [end:jacobi]


# ====================================================================
# Part 4 -- the singular value decomposition
# ====================================================================
```

```python

# [begin:svd_eig]
def svd_via_eig(A, full_matrices=False):
    """SVD from the eigen-decomposition of A^T A (the textbook hand method).

    1. eigenpairs of the smaller Gram matrix (A^T A, or A A^T for wide A)
    2. sigma_i = sqrt(lambda_i)  (clip tiny negative rounding to 0)
    3. u_i = A v_i / sigma_i     (never compute U independently!)
    4. complete U (and V) with orthonormal bases of the null spaces.

    Returns U, s, Vt like numpy.linalg.svd.  Simple, but it squares the
    condition number: singular values below ~ sqrt(eps) * sigma_1 are lost
    (they are returned as 0).
    """
    A = _as_float_matrix(A)
    m, n = A.shape
    if m < n:                                       # wide: work with A^T and swap back
        U, s, Vt = svd_via_eig(A.T, full_matrices)
        return Vt.T, s, U.T
    k = n                                           # now m >= n, so min(m, n) = n
    lam, V = jacobi_eigh(A.T @ A)
    s = np.sqrt(np.clip(lam, 0.0, None))
    # rank decision on the EIGENVALUES: lambda <= max(m,n) eps lambda_1 is noise,
    # i.e. this method cannot resolve sigma below about sqrt(eps) * sigma_1
    tol = max(m, n) * EPS * (lam[0] if lam.size and lam[0] > 0 else 1.0)
    r = int(np.sum(lam > tol))                      # numerical rank
    U = np.zeros((m, r))
    for i in range(r):
        U[:, i] = (A @ V[:, i]) / s[i]
    U = complete_orthonormal_basis(U)               # m x m orthogonal
    s[r:] = 0.0
    if full_matrices:
        return U, s, V.T
    return U[:, :k], s, V.T
# [end:svd_eig]


# [begin:svd_jacobi]
def svd_jacobi(A, full_matrices=False, tol=1e-15, max_sweeps=60):
    """SVD by one-sided Jacobi (Hestenes' method).

    Rotate PAIRS OF COLUMNS of a working copy W = A V until all columns of W
    are mutually orthogonal.  Then W = U Sigma: the column norms are the
    singular values and the normalised columns are the u_i.  A^T A is never
    formed, so small singular values are computed to high relative accuracy.

    For columns w_p, w_q let  alpha = |w_p|^2, beta = |w_q|^2, gamma = w_p.w_q.
    The rotation that makes them orthogonal is the Jacobi rotation that
    diagonalises the 2 x 2 matrix [[alpha, gamma], [gamma, beta]].
    """
    A = _as_float_matrix(A)
    m, n = A.shape
    if m < n:                                       # wide: decompose A^T and swap back
        U, s, Vt = svd_jacobi(A.T, full_matrices, tol, max_sweeps)
        return Vt.T, s, U.T
    W = A.copy()
    V = np.eye(n)
    for _ in range(max_sweeps):
        rotated = False
```

```
        for p in range(n - 1):
            for q in range(p + 1, n):
                alpha = W[:, p] @ W[:, p]
                beta = W[:, q] @ W[:, q]
                gamma = W[:, p] @ W[:, q]
                if abs(gamma) <= tol * np.sqrt(alpha * beta) or gamma == 0.0:
                    continue                    # already orthogonal (to working accuracy)
                rotated = True
                zeta = (beta - alpha) / (2.0 * gamma)
                if abs(zeta) > 1e150:               # avoid overflow in zeta^2
                    t = 0.5 / zeta
                else:
                    t = (1.0 if zeta >= 0 else -1.0) / (abs(zeta) + np.sqrt(1.0 + zeta * zeta))
                c = 1.0 / np.sqrt(1.0 + t * t)
                s_ = c * t
                Wp, Wq = W[:, p].copy(), W[:, q].copy()
                W[:, p] = c * Wp - s_ * Wq
                W[:, q] = s_ * Wp + c * Wq
                Vp, Vq = V[:, p].copy(), V[:, q].copy()
                V[:, p] = c * Vp - s_ * Vq
                V[:, q] = s_ * Vp + c * Vq
        if not rotated:
            break
    s = np.array([norm(W[:, j]) for j in range(n)])
    order = np.argsort(s)[::-1]                 # sort singular values, largest first
    s, W, V = s[order], W[:, order], V[:, order]
    tol_rank = max(m, n) * EPS * (s[0] if s.size and s[0] > 0 else 1.0)
    r = int(np.sum(s > tol_rank))
    U = np.zeros((m, r))
    for i in range(r):
        U[:, i] = W[:, i] / s[i]
    U = complete_orthonormal_basis(U)           # fill in u_{r+1}, ..., u_m
    s[r:] = 0.0
    if full_matrices:
        return U, s, V.T
    return U[:, :n], s, V.T


def svd(A, full_matrices=False):
    """Our default SVD: one-sided Jacobi."""
    return svd_jacobi(A, full_matrices=full_matrices)
# [end:svd_jacobi]


# ============================================================================
# Part 5 -- the pseudo-inverse
# ============================================================================

# [begin:pinv]
def pinv(A, rcond=None):
    """Moore-Penrose pseudo-inverse  A^+ = V Sigma^+ U^T.

    Singular values <= rcond * sigma_1 are treated as zero.
    Default rcond = max(m, n) * eps  (the same threshold as matrix rank).
    """
    A = _as_float_matrix(A)
    m, n = A.shape
    U, s, Vt = svd(A)
    if rcond is None:
```

```
        rcond = max(m, n) * EPS
    if s.size == 0 or s[0] == 0.0:
        return np.zeros((n, m))
    keep = s > rcond * s[0]
    s_inv = np.zeros_like(s)
    s_inv[keep] = 1.0 / s[keep]
    # V Sigma^+ U^T = sum_i (1/sigma_i) v_i u_i^T ; broadcasting scales the columns of V
    return (Vt.T * s_inv) @ U.T


def pinv_full_rank(A):
    """Pseudo-inverse from the full-rank formulas (no SVD needed).

    independent columns:  A^+ = (A^T A)^{-1} A^T      (left inverse)
    independent rows:     A^+ = A^T (A A^T)^{-1}      (right inverse)
    """
    A = _as_float_matrix(A)
    m, n = A.shape
    r = rank_by_elimination(A)
    if r == n:
        return solve(A.T @ A, A.T)
    if r == m:
        return solve(A @ A.T, A).T             # (A A^T)^{-1} is symmetric
    raise ValueError("A is rank deficient: use the SVD-based pinv")


def penrose_residuals(A, X):
    """Frobenius norms of the four Penrose-condition residuals (all ~0 for X = A^+)."""
    A = _as_float_matrix(A)
    X = _as_float_matrix(X)
    AX, XA = A @ X, X @ A
    fro = lambda M: float(np.sqrt(np.sum(M * M)))
    return (fro(A @ X @ A - A), fro(X @ A @ X - X), fro(AX - AX.T), fro(XA - XA.T))
# [end:pinv]


# ======================================================================
# Part 6 -- least squares
# ======================================================================

# [begin:cholesky]
def cholesky(S):
    """Cholesky factor L (lower triangular) with S = L L^T, for S positive definite."""
    S = _as_float_matrix(S)
    n = S.shape[0]
    L = np.zeros((n, n))
    for j in range(n):
        d = S[j, j] - L[j, :j] @ L[j, :j]
        if d <= 0.0:
            raise ValueError("matrix is not positive definite")
        L[j, j] = np.sqrt(d)
        L[j + 1:, j] = (S[j + 1:, j] - L[j + 1:, :j] @ L[j, :j]) / L[j, j]
    return L
# [end:cholesky]


# [begin:lstsq]
def lstsq_normal(A, b):
    """Least squares via the normal equations A^T A x = A^T b (Cholesky solve)."""
```

```
    A = _as_float_matrix(A)
    L = cholesky(A.T @ A)
    y = forward_substitution(L, A.T @ np.asarray(b, dtype=float))
    return back_substitution(L.T, y)


def lstsq_qr(A, b):
    """Least squares via Householder QR:  R_1 x = (Q^T b)_1."""
    A = _as_float_matrix(A)
    n = A.shape[1]
    Q, R = householder_qr(A)
    return back_substitution(R[:n, :n], (Q.T @ np.asarray(b, dtype=float))[:n])


def lstsq_svd(A, b, rcond=None):
    """Minimum-norm least-squares solution  x = sum_i (u_i^T b / sigma_i) v_i = A^+ b."""
    return pinv(A, rcond) @ np.asarray(b, dtype=float)
# [end:lstsq]


# [begin:regularise]
def tsvd_solve(A, b, k, usv=None):
    """Truncated-SVD solution: keep only the k largest singular values.

    Pass usv=(U, s, Vt) to reuse an SVD you already computed (e.g. when
    trying many values of k).
    """
    U, s, Vt = svd(A) if usv is None else usv
    c = U[:, :k].T @ np.asarray(b, dtype=float)       # coefficients u_i^T b
    return Vt[:k].T @ (c / s[:k])


def ridge_solve(A, b, lam, usv=None):
    """Tikhonov / ridge solution  x = sum_i sigma_i/(sigma_i^2 + lam) (u_i^T b) v_i."""
    U, s, Vt = svd(A) if usv is None else usv
    c = U.T @ np.asarray(b, dtype=float)
    return Vt.T @ (s / (s ** 2 + lam) * c)
# [end:regularise]


def vandermonde(t, degree):
    """Columns 1, t, t^2, ..., t^degree (for polynomial fitting)."""
    t = np.asarray(t, dtype=float)
    return np.column_stack([t ** j for j in range(degree + 1)])


# ======================================================================
# Part 7 -- applications
# ======================================================================

# [begin:norms]
def singular_values(A):
    return svd(A)[1]


def spectral_norm(A):
    s = singular_values(A)
    return float(s[0]) if s.size else 0.0

```

```python

def frobenius_norm(A):
    A = _as_float_matrix(A)
    return float(np.sqrt(np.sum(A * A)))


def nuclear_norm(A):
    return float(np.sum(singular_values(A)))


def condition_number(A):
    """kappa = sigma_1 / sigma_r (smallest NON-ZERO singular value); inf if A = 0."""
    s = singular_values(A)
    s = s[s > max(np.shape(A)) * EPS * (s[0] if s.size else 0.0)]
    return float(s[0] / s[-1]) if s.size else np.inf


def matrix_rank(A, tol=None):
    s = singular_values(A)
    if s.size == 0:
        return 0
    if tol is None:
        tol = max(np.shape(A)) * EPS * s[0]
    return int(np.sum(s > tol))
# [end:norms]


# [begin:lowrank]
def low_rank_approx(A, k):
    """Best rank-k approximation (Eckart-Young): sum_{i<=k} sigma_i u_i v_i^T."""
    U, s, Vt = svd(A)
    k = min(k, s.size)
    return (U[:, :k] * s[:k]) @ Vt[:k]


def pca(X, k):
    """Principal component analysis of the rows of X (N samples x d features).

    Returns (components (k x d), explained_variance (k,), scores (N x k), mean).
    """
    X = _as_float_matrix(X)
    mean = X.mean(axis=0)
    Xc = X - mean                                  # centre the data
    U, s, Vt = svd(Xc)
    N = X.shape[0]
    explained_variance = s[:k] ** 2 / (N - 1)
    scores = U[:, :k] * s[:k]                      # = Xc @ Vt[:k].T
    return Vt[:k], explained_variance, scores, mean
# [end:lowrank]
```

APPENDIX B

Helper Scripts

plotstyle.py

```python
"""Shared Matplotlib style for the Lab Manual figures (import it before plotting)."""
import os
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

# categorical colours, always used in this order
C1, C2, C3 = "#2a78d6", "#eb6834", "#1baf7a"
INK, INK2, GRID = "#0b0b0b", "#52514e", "#e4e3df"

plt.rcParams.update({
    "figure.figsize": (6.0, 3.4),
    "figure.dpi": 150,
    "font.size": 9,
    "axes.edgecolor": INK2,
    "axes.labelcolor": INK,
    "axes.titlesize": 10,
    "axes.titleweight": "bold",
    "axes.spines.top": False,
    "axes.spines.right": False,
    "axes.grid": True,
    "grid.color": GRID,
    "grid.linewidth": 0.6,
    "xtick.color": INK2,
    "ytick.color": INK2,
    "lines.linewidth": 1.6,
    "lines.markersize": 4.5,
    "legend.frameon": False,
    "savefig.bbox": "tight",
})


def savefig(name):
    """Save the current figure as figs/<name>.pdf and close it."""
    os.makedirs("figs", exist_ok=True)
    plt.savefig(os.path.join("figs", name + ".pdf"))
    plt.close()
```

run_demos.py

```
"""Run every demo script and capture its output in out/<name>.txt (figures go to figs/)."""
import glob, os, subprocess, sys, time
here = os.path.dirname(os.path.abspath(__file__))
os.makedirs(os.path.join(here, "out"), exist_ok=True)
env = dict(os.environ, PYTHONPATH=here)
for path in sorted(glob.glob(os.path.join(here, "demos", "d*.py"))):
    name = os.path.splitext(os.path.basename(path))[0]
    t0 = time.time()
    r = subprocess.run([sys.executable, path], cwd=here, env=env, capture_output=True, text=True)
    with open(os.path.join(here, "out", name + ".txt"), "w") as f:
        f.write(r.stdout.rstrip() + "\n")
    status = "ok" if r.returncode == 0 else "FAILED"
    print(f"{name:22s} {status}  ({time.time() - t0:.1f}s)")
    if r.returncode:
        print(r.stderr)
```